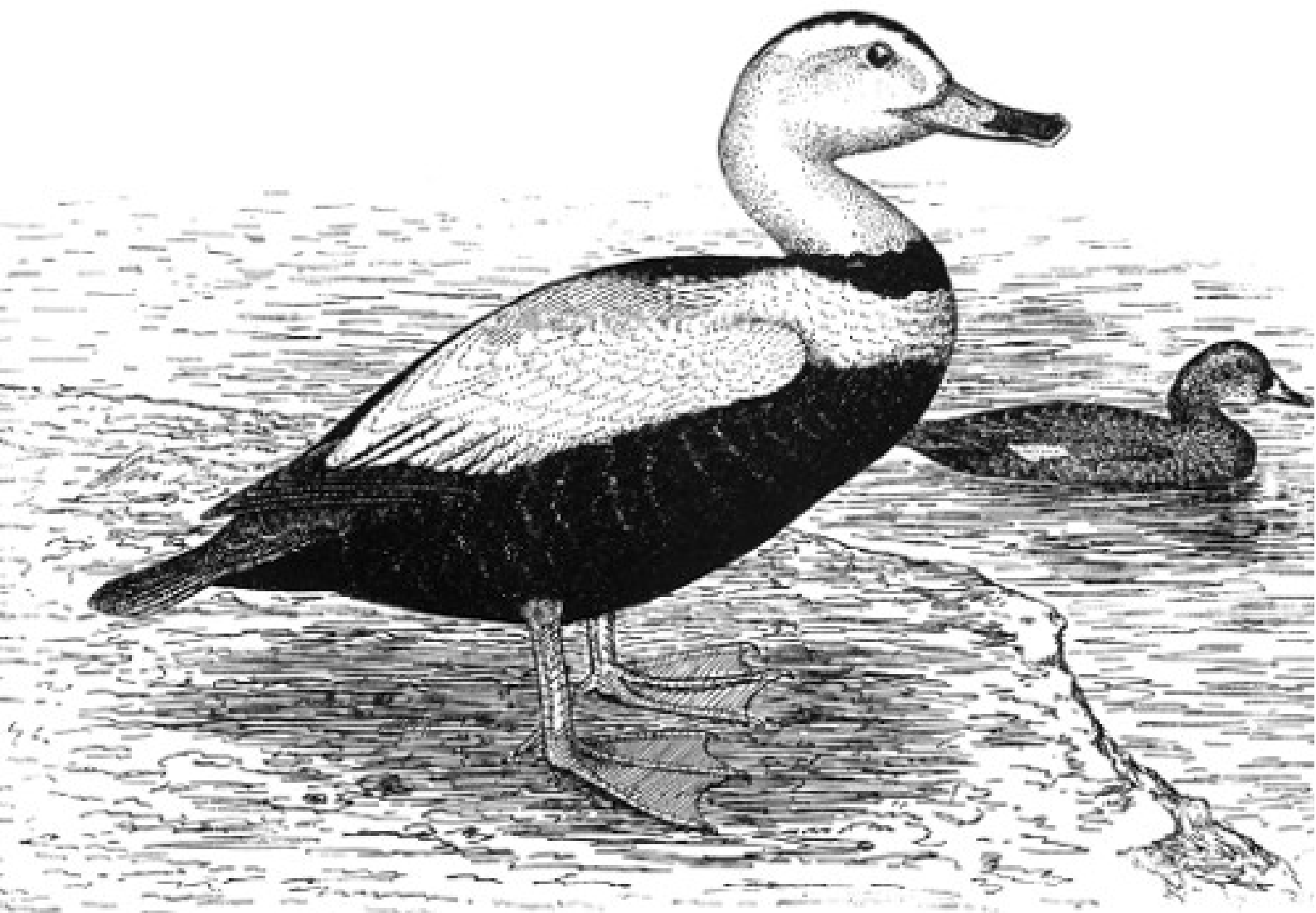


Xms 5Gb for your HelloWorld



Programming

Learning Java

O RLY?

Pismak A.E.

Немного слов, прежде чем Вы коснетесь этого...

Конспект лекций по курсу «Программирование» построен на базе тех слайдов и «басней», которые ты, дорогой читатель, слышал на лекционных занятиях.

Курс первого семестра по дисциплине «Программирование» состоит из одной вводной лекции и семи содержательных. В рамках лекций первого семестра мы постараемся изучить основы программирования на языке Java, а именно изучим его основные синтаксические конструкции, реализацию объектно-ориентированных парадигм и принципов на примере этого языка. В первом семестре нам также предстоит изучить некоторые языковые инструменты, такие как «Исключения» и «Перечисления» и т. п. Помимо теоретических основ, в рамках лабораторных занятий, Вам будет предложено реализовать несколько программ, тем самым демонстрируя полученные Вами знания на практике.

Этот конспект разбит на несколько разделов — для каждой лекции по разделу. Каждый раздел представляет собой слайд из лекции и комментарии к этому слайду, которые тезисно излагают содержимое лекции.

В этом конспекте помимо комментариев можно встретить различные конструкции, которые указывают на подразделы для самостоятельного изучения, либо указывают на инструментарий, которым рекомендуется пользоваться при самостоятельной работе. Такие подраздела, как правило отделяются чертой, вида:

НАЗВАНИЕ РАЗДЕЛА

Прежде чем Вы начнете, ловите несколько советов:

- Обратного пути нет. Все мы по началу думаем, что сможем завязать с программированием...
- Весь материал, который Вы понимаете, сразу применяйте на практике. Придумывайте идеи и старайтесь реализовывать, используя то, чему научились на занятиях или при самостоятельном изучении.
- Весь материал, который Вы НЕ поняли, ~~выбросьте в окно~~ как можно быстрее переспросите или попросите объяснить кого-либо, кто понял. Это важно! Каждая следующая тема зависит от предыдущей. Отстав однажды, можно уже не догнать.
- Преподаватель всегда на Вашей стороне. У него нет ни желания, ни мотивов Вас завалить. Сотрудничайте с преподавателями.
- Преподаватель — человек. Ему свойственно что-либо забыть или не знать. Это нормально. Поиск ошибок в его презентациях и этом конспекте только повысит Ваши баллы.

Успехов!



УНИВЕРСИТЕТ ИТМО

Кафедра вычислительной техники

Программирование

Лекция #0. Введение.

Знакомство и немного о курсе:

- Для чего этот курс?
- Что будем изучать?
- Какие практические задания будем выполнять?



Notes

Теоретическая часть курса включает в себя изучение основ программирования на примере языка программирования Java, а именно: основные конструкции языка и их назначение, объектно-ориентированная модель программирования и ее реализация в Java, структуры хранения данных, а также алгоритмы и типовые приемы решения проблем проектирования программного обеспечения.

В практическую часть курса входят лабораторные работы, в ходе которых студентам предлагается написать простые программы, демонстрирующие приобретенные знания и навыки студента.

Организационные вопросы

- Где брать материалы и задания?
- Где смотреть свои текущие оценки?
- БАРС
 - Какие и сколько будет лабораторных
 - Какие и сколько будет контрольных
 - Зачеты / Экзамены



3

Notes

БАРС (Бально рейтинговая система) — система контроля знаний и оценки успеваемости студентов Университета ИТМО. Подразумевает оценку работы студента баллами в течении семестра, по завершению которого сумма полученных за семестр баллов формирует оценку студента. Предусмотрено получение баллов за следующие виды работ:

МОДУЛЬ 1	Лабораторные работы (3 шт)
	Контрольные (лекционные 4 шт)
	Рубежная контрольная
МОДУЛЬ 2	Лабораторные работы (2 шт)
	Контрольные (лекционные 4 шт)
	Рубежная контрольная
СЕССИЯ	Зачет

Текущие оценки: см. ленту на <https://helios.cs.ifmo.ru/>

Материалы и задания: см. раздел ПРОГРАММИРОВАНИЕ на <https://helios.cs.ifmo.ru/>

Рекомендуемый материал

- Гаврилов А.В., Клименков С.В., Цопа Е.А.
“Программирование на Java. Конспект лекций”
- Кей Хорстманн, Гари Корнелл “Java. Библиотека профессионала. Том 1. Основы”
- Кей Хорстманн “Java SE 8. Вводный курс” (“Java 8 для нетерпеливых”)

Notes

Источники материалов:

- <https://se.ifmo.ru/>
- Библиотека
- Эти «ихние» интернеты

Инструменты для выполнения лабораторных работ

- Текстовые редакторы
- JDK
- IDE

Notes

IDE (Integrated Development Environment) — пакет программных средств, которые предоставляют программисту удобный интерфейс для компиляции, запуска и отладки программ.

JDK (Java Development Kit) — пакет программных инструментов, которые предоставляют программисту средства для компиляции, запуску, отладке своих программ. В отличие от сред разработки (IDE), JDK предоставляет только инструменты и не предоставляет средств для автоматизации процессов в жизненном цикле программного обеспечения. При использовании JDK, исходный код программы необходимо подготовить в каком-либо текстовом редакторе.

ВНИМАНИЕ! ПЕРВЫЕ 3 (ТРИ) ЛАБОРАТОРНЫЕ РАБОТЫ НЕОБХОДИМО ВЫПОЛНИТЬ ИСПОЛЬЗУЯ СРЕДСТВА JDK!

Предметная область

- Что такое программа?
- Что такое программирование?

Notes

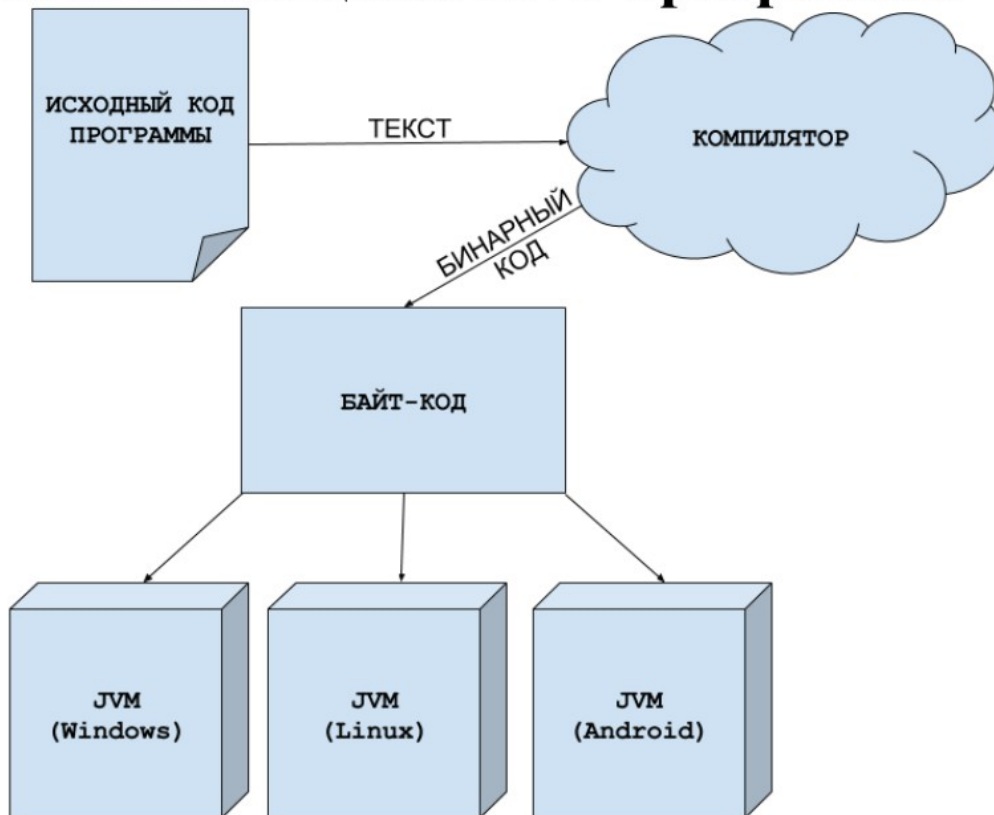
Программа — перечень инструкций, в совокупности определяющий алгоритм решения некоторой задачи. Инструмент автоматизации повторяющихся действий или ручного труда человека.

Программирование — процесс реализации программ.

Задачи программиста:

- исследование предметной области
- проектирование программного обеспечения
- реализация
- оптимизация (при необходимости)

Жизненный цикл Java-программы



Основы программирования. ИТМО ВТ.

Лекция #0

8

Notes

Жизненный цикл разработки программ:

НАПИСАНИЕ ИСХОДНОГО КОДА →
→ КОМПИЛЯЦИЯ В МАШИННЫЙ КОД →
→ ВЫПОЛНЕНИЕ

Жизненный цикл разработки Java-программ:

НАПИСАНИЕ ИСХОДНОГО КОДА →
→ КОМПИЛЯЦИЯ В БАЙТ КОД →
→ ЗАПУСК JVM → ВЫПОЛНЕНИЕ

Причина наличия такого жизненного цикла: кроссплатформенность получаемого ПО.

Компилятор

```
public class Alp
  public static
    x = y * 2;
  }
}
```

```
...
load_const 2
load y
mul y, 2
save $0
...
```

Notes

Рассмотрим два основных этапа программы на языке Java. Первый из них — компиляция. В классических языках программирования на этом этапе происходит формирование машинных кодов, которые могут быть выполнены на той операционной системе и на том процессоре, под которым программа была скомпилирована. В случае с Java это выглядит несколько иначе. Компилятор формирует некоторую последовательность элементарных инструкций (так называемый byte-код) на промежуточном языке. Полученный в итоге компиляции файл с этим кодом не может быть исполнен, так как содержит двоичные данные непонятные платформе (операционной системе и центральному процессору).

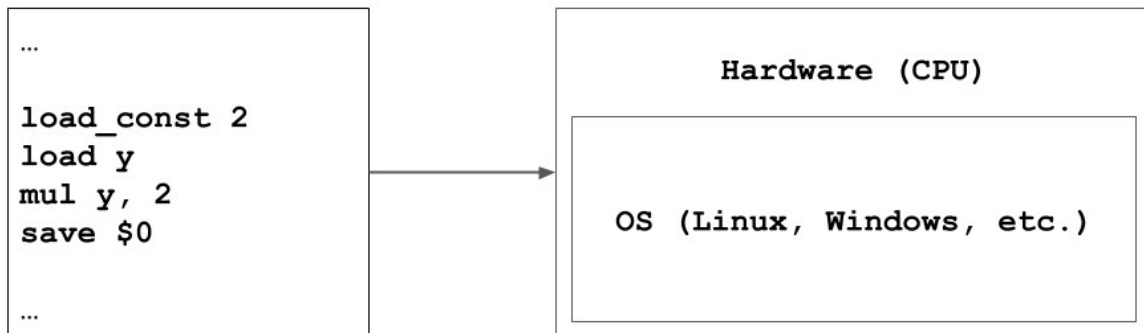
На слайде приведен пример того, как может выглядеть фрагмент исходного кода и промежуточного byte-кода.

ИНСТРУМЕНТАРИЙ

Для компиляции исходного кода программы на языке программирования Java Вы можете использовать утилиту *javac*. Для просмотра команд полученного byte-кода может быть использована команда *javap* с ключем *-c*.

```
javac Main.java
javap -c Main.class
```

JVM (Java Virtual Machine)



Гвоздь программы -
КРОССПЛАТФОРМЕННОСТЬ

НИЧОСИ



Основы программирования. ИТМО ВТ.

Notes

Следующий этап — это запуск и выполнение полученного byte-кода. Полученный промежуточный код является стандартизованным и имеет одинаковый формат не зависимо от того на какой платформе он будет выполнен. Так же стоит отметить, что результат выполнения одного и того же кода на разных платформах будет идентичен. За счет чего все это достигается?

В этот момент самое время выйти на сцену такому инструменту языка, как виртуальная машина Java (JVM, Java Virtual Machine). Это программа, которая написана разработчиками языка Java для каждой платформы. Ее функция заключается в том, чтобы выполнить команды указанные в промежуточном byte-коде, интерпретируя их в соответствующие машинные команды (понятные аппаратному и программному обеспечению компьютера).

САМОСТОЯТЕЛЬНО

Для ускорения работы по интерпретации команд byte-кода используется так называемый JIT-компилятор (Just In Time). Полезным будет изучить его назначение и принципы работы более подробно.

ИНСТРУМЕНТАРИЙ

Запуск промежуточного кода осуществляется утилитой java:

```
java Main
```

Состав языка

- Символы
- Лексемы
- Выражения
- Операторы

Notes

Символы — набор знаков, допустимых для словообразования (алфавит) в данном языке. Применимо к языкам программирования — это множество знаков, которые могут быть использованы при написании исходного кода программы.

Лексемы — слова, образованные из алфавита заданного языка. Являются минимальной смысловой единицей в тексте. Применимо к языкам программирования — это последовательности символов, являющиеся именами переменных, ключевыми словами языка либо специальными или зарезервированными последовательностями символов.

Выражения — множество слов (или лексем), которые несут в себе некоторую законченную мысль. В естественных языках применим термин — предложения. В языках программирования примером предложения (выражения) может быть следующее:

$$x = v * t + x0;$$

Операторы — в общем случае, служебные элементы логического языка. В естественных языках к ним можно отнести артикли. Применимо к языкам программирования — это служебные последовательности символов, применяемые к одному или нескольким операндам в составе выражений.

Зарезервированные лексемы

`abstract` `continue` `for` `new` `switch` `assert` `default`
`goto` `package` `synchronized` `boolean` `do` `if` `private`
`this` `break` `double` `implements` `protected` `throw`
`byte` `else` `import` `public` `throws` `case` `enum`
`instanceof` `return` `transient` `catch` `extends` `int`
`short` `try` `char` `final` `interface` `static` `void`
`class` `finally` `long` `strictfp` `volatile` `const`
`float` `native` `super` `while`

Notes

Подчеркнуты и выделены зеленым цветом ключевые слова языка программирования Java, которые будут изучены студентами в течении этого семестра.

Черным выделены ключевые слова Java, которые, возможно, будут затронуты в этом семестре, но в рамках курса либо будут изучены во втором семестре, либо не входят в рамки этого курса.

Красным выделены ключевые слова, которые зарезервированы в Java, но не имеют практического применения и являются неиспользуемыми.

Комментарии в программах Java

Однострочные комментарии

```
// комментарии
```

Многострочные комментарии

```
/*  
комментарии  
*/
```

Специальные комментарии (javadoc)

```
/**  
 *  
 * @author James Gosling  
 */
```

Notes

Комментарии — это конструкции языка, которые необходимы для введения в исходный код программы строк текста, игнорируемых компилятором.

Специальные комментарии необходимы для автоматического формирования документации к исходному коду с помощью утилиты `javadoc`, входящей в состав JDK.

ИНСТРУМЕНТАРИЙ

Для автоматического формирования документации по исходным кодам программы используется утилита `javadoc`, входящая в состав JDK. В качестве аргумента утилита принимает путь к файлам с исходными кодами. Необязательным параметром можно указать директорию, куда будет собрана вся полученная документация:

```
javadoc путь_к_java_исходникам -d директория_для_документации
```

“Привет мир” на языке Java

HelloWorld.java

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello, world!");  
  
    }  
  
}
```

Notes

В данном примере, строка:

```
System.out.println("Hello, world!");
```

осуществляет вывод сообщения на стандартный поток вывода. Остальную конструкцию кода, будем пока считать каркасом любой программы на Java, которые будем реализовывать в рамках лабораторной работы.

Имя файла в котором находится этот исходный код должен иметь следующий формат:

имя_класса.java

где имя_класса — это имя, которое мы указали в исходном коде после ключевого слова **class**

Простая арифметическая программа

```
public class SimpleProgram {  
    public static void main(String[] args) {  
        double x = 5.0D;  
        double result = Math.pow(x, 2.5);  
        System.out.println(result);  
    }  
}
```

Notes

Разберем по строкам, за исключением конструкции, упомянутой в предыдущем слайде.

1. Объявление вещественной переменной двойной точности с присвоением ей значения 5.
2. Объявление вещественной переменной с присвоением ей результата математической операции возведения в степень.
3. Вывод результата.



Кафедра вычислительной техники

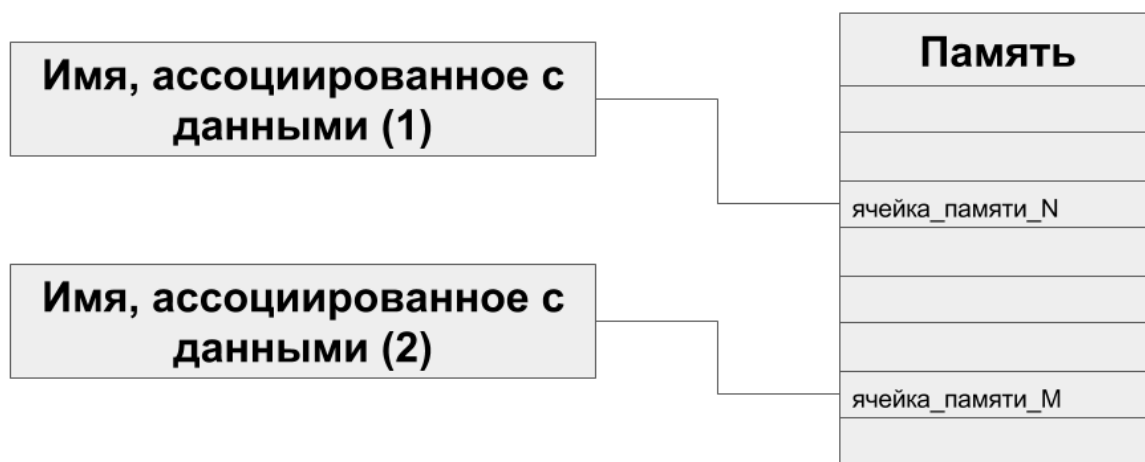
Программирование

Лекция #1.

Типы данных. Строки.

Математические функции.

Представление данных



Notes

Любую программу можно представить в виде данных и кода, который осуществляет обработку этих данных. Взаимодействие с данными в исходном тексте программы осуществляется посредством обращения к участку памяти в котором хранятся необходимые данные. При этом для каждого такого фрагмента данных имеется некоторый именованный указатель, называемый *именем переменной* или *именем константы*.

Пример. Память можно представить в виде тетрадного листа. Каждая клетка — ячейка памяти. В каждой ячейке может храниться некоторое значение (число, символ и т. п.). Для каждой ячейки в которой находится значение, назначается имя по которому из исходного текста программы мы можем получить доступ к значению. Таким образом, организовано взаимодействие кода с данными.

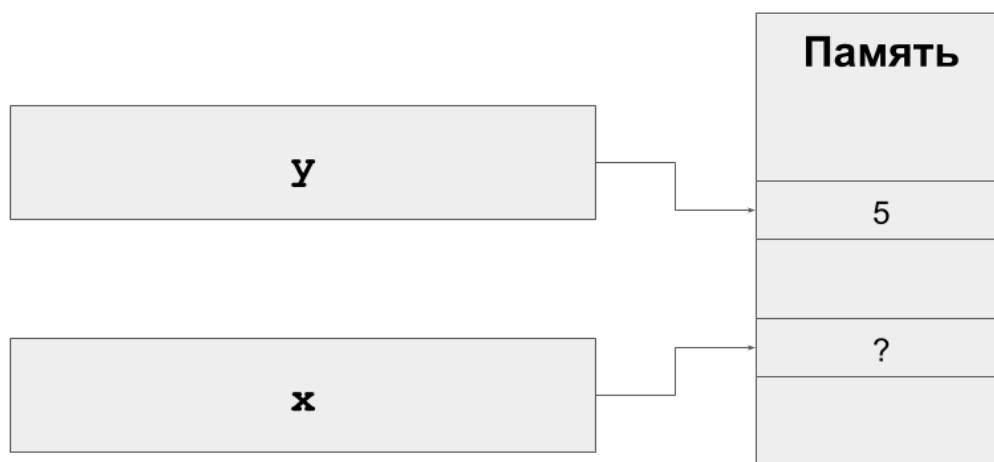
О том, что из себя представляют переменные и константы мы рассмотрим далее.

Переменные

```
int x;
```

```
int y = 5;
```

```
тип_данных имя_переменной [ = значение_по_умолчанию ] ;
```



5

Notes

Переменная — это, как уже ранее оговаривалось, некоторый именованный указатель на фрагмент памяти, содержащий значение необходимых данных.

На слайде приведено два примера объявления переменных. Общий шаблон синтаксиса объявления переменных представлен в строке:

```
тип_данных имя_переменной [ = значение_по_умолчанию ] ;
```

о том, каких типов могут быть данные будет обсуждаться позже. Сейчас необходимо обсудить требования, предъявляемые к именам переменных, а так же о роли значений по умолчанию и правила использования переменных. Имя переменной НЕ должно начинаться с цифры, но цифры могут быть использованы в формировании имени переменной. Имя переменной может начинаться и содержать в себе латинские символы либо символа '_' . Переменная, которая объявлена без присваивания ей значения по умолчанию не может быть использована, пока ей не будет присвоено значение (на слайде — это переменная x). Значение переменной может меняться по ходу выполнения программы (отсюда и название термина — *переменная*).

Константы

Объявление констант

```
final double E = 5.0;
```

```
final double R;  
// ...  
R = 5.0;
```

Назначение констант

```
int total = 8 * 5 * 4 * 12; // ПЛОХО
```

```
final int HOURS = 8, DAYS = 5, WEEKS = 4, MONTHS = 12;  
// ...  
int total = HOURS * DAYS * WEEKS * MONTHS; // ХОРОШО
```

6

Notes

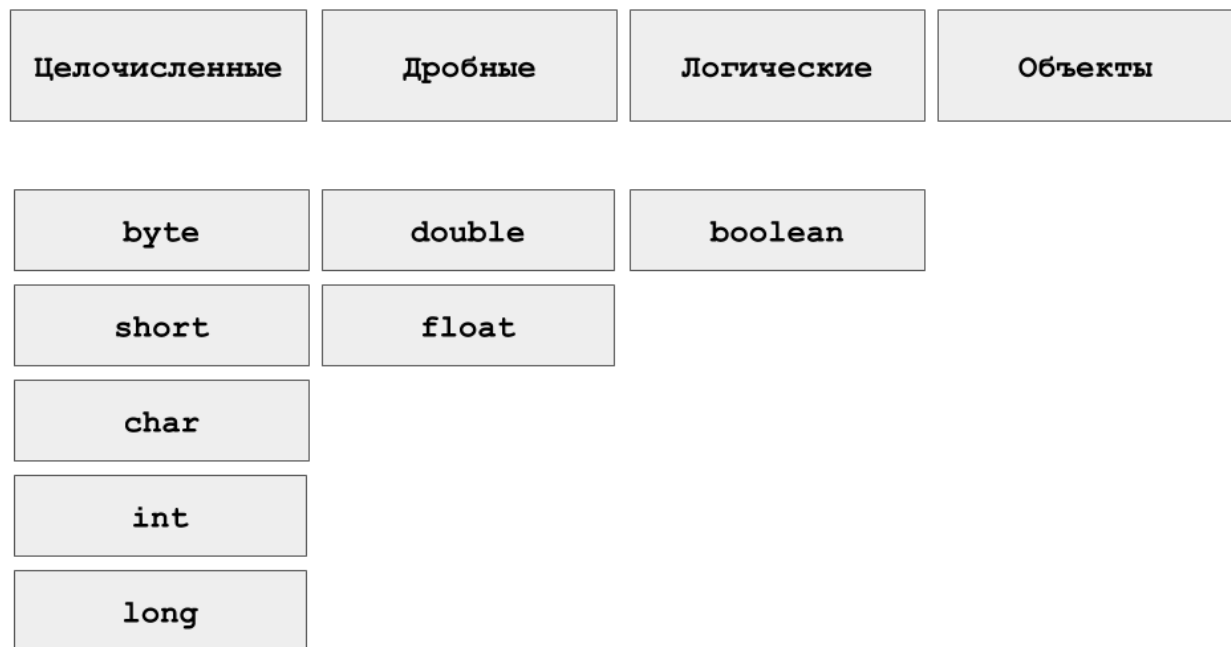
Если есть необходимость в использовании данных, значение которых не будет меняться в ходе выполнения программы, то в таких случаях применимы *константы*.

Константы — это, как и переменные, именованные указатели на значения хранимые в памяти. Разница в том, что эти значения находятся в той области памяти, которые допустимы для разовой записи. Т.е. присвоение значений константам можно произвести только один раз за все время выполнения программы.

Синтаксис объявления констант схож с объявлением переменных с одной только разницей — в объявлении константы, перед указанием типа данных указывается ключевое слово *final*. Примеры объявления констант представлены на слайде. Хорошим тоном именования констант считается использование заглавных букв, а для разделения слов в имени константы использование знака '_'.

Использование в тексте программы не именованных значений является плохой практикой, т. к. в значительной степени снижает показатели по пониманию и сопровождению исходного кода программы. Пример использования констант представлен на слайде. В случае расчета количества рабочих часов в год без использования констант, далеко не очевидно, что обозначают те или иные числа, используемые при расчетах. Однако при использовании осмысленных имен констант для числовых значений, смысл исходного кода становится понятен.

Типы данных. Классификация.



7

Notes

Все типы данных можно классифицировать таким образом, как показано на слайде. Первые две категории — это числа, разница между которыми заключается только в количестве памяти, которые они занимают. Можно отметить, что, так называемый «символьный» тип *char* тоже относится к целочисленному классу типов данных.

Логический тип — это вариант данных, значения которых могут принимать только два значения: *true* или *false*. Это удобно в тех случаях, когда необходимо хранить данные некоторых битовых флагов или признаков. Конкретные примеры: хранение информации о том, нажата ли кнопка и т. п.

Объекты — это сложный класс данных, которые будут рассматриваться в рамках данного курса несколько позднее.

Типы данных. Подробности.

Тип данных	Размер (бит)	Значения
byte	8	от -128 до 127
short	16	от -32768 до 32767
char	16	от 0 до 65535
int	32	от -2147483648 до 2147483647
long	64	от -9223372036854775808 до 9223372036854775807
float	32	от -1.4e-45f до 3.4e+38f
double	64	от -4.9e-324 до 1.7e+308
boolean	1 или 32	true или false

Notes

В таблице представлены характеристики стандартных типов данных, а именно количество потребляемой памяти и диапазон допустимых значений.

Особое внимание стоит обратить на логический тип (*boolean*). Память, потребляемая для хранения значений этого типа варьируется в зависимости от контекста, в котором определена переменная этого типа. При использовании массивов типа *boolean*, для хранения одного значения используется один бит данных. При объявлении самостоятельной переменной этого типа, хранимое значение потребляет 32 бита памяти (буквально, используется тип *int*).

Еще особенностью языка является тип *char*. Как может показаться из названия, а также опыт работы в других языках, что это «символьный» тип данных. Отчасти, так и есть, но Java специфицирует его целочисленный тип данных, а именно, как «беззнаковый *short*», т. к. *short* - это 16-битный тип, а для хранения кода символа в кодировке Unicode нужно именно столько.

Использование “больших чисел”

Целые числа

```
BigInteger bigInt = new
BigInteger("99999999999999999999999999999999");

BigInteger one = new BigInteger("1");

BigInteger result = bigInt.add(one);
```

Дробные числа

```
BigDecimal bigFloat = new  
BigDecimal("9999999999999999.9999999999999999");  
  
BigDecimal one = new BigDecimal("0.000000000000000001");  
  
BigDecimal result = bigFloat.add(one);
```

9

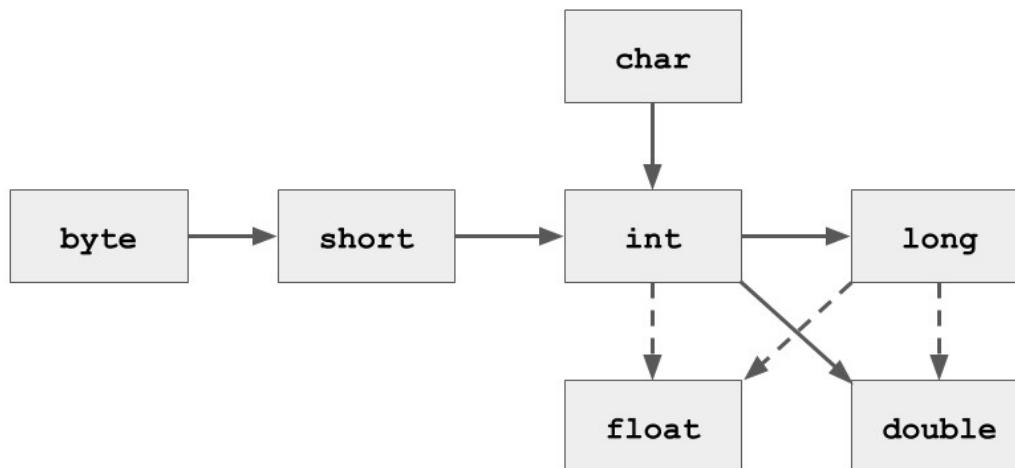
Notes

Возникают случаи, когда необходимо использовать числа, разрядность которых превышает *long* и *double*. Например для расчетов в астрономии, т. к. расстояния между космическими телами достигают миллиардов световых лет, то перевод подобных чисел в систему СИ (т. е. в метры) приводит к потере значащих знаков.

Для решения подобного рода задач в Java есть пара инструментов: `BigInteger` — для больших целых чисел, и `BigDecimal` — для больших дробных чисел.

Механизм работы с такими инструментами представлен на слайде. В рамках первых лабораторных работ с этими механизмами сталкиваться не придется. В данной лекции мы говорим о них только для ознакомления.

Преобразование типов



10

Notes

Преобразование типов — это возможность работать со значением в памяти одного типа, как со значением другого. Например,

```
short x1 = 255, x2 = 18;

int currentValue = x1;

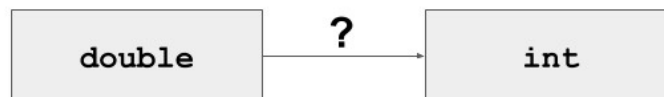
// some code

currentValue = x2;
```

в данном случае мы объявили переменные типа *short*, но в дальнейшем использовали их значения как тип *int*. Этот механизм и называется преобразованием типов. Преобразование без потерь возможно лишь в том случае, если преобразуемое значение по своей размерности помещается в разрядную сетку целевого типа (в который преобразовывается). Так, тип *int* вмещает в себя все значения типа *short* и по этому может быть преобразован без потерь. На рисунке преобразования без потерь показаны сплошными стрелками. Пунктирными стрелками обозначены преобразования с возможными потерями.

Иные преобразования (не обозначенные стрелками) в Java не предусмотрены, но есть механизм приведения типов для обхода этого ограничения.

Привидение типов



```
double e = 2.7;
```

```
int notE = (int) e;    // notE = 2
```

```
int roundE = (int) Math.round(e);    // roundE = 3
```

13

Notes

Преобразования типов — это механизм конвертации типов, не предусмотренный стандартным преобразованием. Например, Вы посчитали высоту стола (92,53 см), но вам не важна точность и для дальнейших расчетов Вы намерены использовать только целую часть числа, тогда может быть использована следующая конструкция:

```
double height = 92,53;
```

```
int totalHeight = (int) height;
```

Для использования округления необходимо использовать конструкцию, представленную на слайде.

Операции

Унарные

-
--
++
~
!
[]

Бинарные

- +
/ *
| ||
& &&
== !=

```
int x = -y;           // унарный "минус"
```

```
int r = x - y;        // бинарный "минус"
```

12

Notes

Унарные операторы — это операторы выполняющие операцию над одним аргументом. Например унарный минус осуществляет инвертирование знака числа:

```
x = -x;
```

Операции уменьшения и увеличения числа на единицу:

```
x++; ++x;  
y--; --y;
```

Бинарные операции — это операции с участием двух переменных, например бинарный минус осуществляет вычитание одного аргумента из другого, оператор присваивания осуществляет вычисление выражения, стоящего справа от него и присваивает переменной слева от него и т. д. :

```
int result = x - y;
```

```
int square = height * width;
```

Приоритеты операций

№	Оператор
1	[] . ()
2	! ~ ++ -- + - (приведение) new
3	* / %
4	+ - (бинарные)
5	>> << >>>
6	< <= > >= instanceof
7	== !=
8	&
9	^
10	
11	&&
12	
13	?:
14	= += -= *= /= %= = ^= <<= >>= >>>=

13

Notes

Операторы расположены в порядке убывания приоритетов.

Математические функции

Функция	Конструкция
Абсолютное значение x	<code>Math.abs (x);</code>
Косинус x	<code>Math.cos (x);</code>
Синус x	<code>Math.sin (x);</code>
Экспонента	<code>Math.exp (x);</code>
Квадратный корень x	<code>Math.sqrt (x);</code>
Корень суммы квадратов (гипотенуза)	<code>Math.hypot (x, y);</code>
Натуральный логарифм	<code>Math.log (x);</code>
Округление до ближайшего	<code>Math.round (x);</code>
Возведение ' x ' в степень ' y '	<code>Math.pow(x, y);</code>

<https://docs.oracle.com/javase/7/docs/api/java/lang/Math.html>

14

Notes

В рамках первой лабораторной работы потребуется закодировать некоторую математическую формулу. При этом необходимо использовать стандартные математические функции Java. Правило записи этих функций для получения конкретного результата имеет следующий вид:

```
result = Math.имя_функции ( аргументы_через_запятую );
```

result в данном случае может быть разного типа, в зависимости от использованной функции.

Математические константы

- `Math.PI` `// 3.14159265358979323846`
- `Math.E` `// 2.7182818284590452354`

```
double res1 = 3 * Math.PI / 2;
```

```
double res2 = Math.log(5) * Math.E;
```

15

Notes

Для использования в математических формулах таких констант, как числа *PI* и *E* нет необходимости объявлять эти константы в своих исходных текстах. Эти константы объявлены в `Math` и примеры их использования представлены на слайде.

Строки

```
String name = "Alis";
```

```
String lastName = new String("Parker");
```

```
String a = "Java\u2122";           // Java™  
String b = "";                   // пустая строка
```

<http://docs.oracle.com/javase/6/docs/api/java/lang/String.html>

16

Notes

Строки — это второй по степени интерактивности инструмент взаимодействия пользователя с программой после изображений. С помощью текстовой информации программы предоставляет пользователю приглашения для ввода данных, а также предоставляет все результаты своей работы. Строка в Java — это последовательность символов Unicode. (В Java 9 — это будет изменено, но до Java версии 1.8 включительно, строка остается именно такой последовательностью).

На слайде представлены примеры объявления строковых переменных. Строка — это не примитивный тип и по сути своей является объектом, о которых мы говорили в разделе классификации типов данных. Строки не являются изменяемым типом, т. е. Хранимую в памяти строку нельзя поменять. Если мы хотим добавить к ней символ, то нам придется выделить новый фрагмент памяти, размеров с нашу строку плюс добавляемый символ, затем скопировать нашу строку в новый фрагмент памяти и добавить в конец необходимый символ. Именно этот процесс происходит при конкатенации строк, которую мы рассмотрим далее.

Как известно символы Unicode представляются в виде последовательности **\u** и **кода символа**. По этому для Java не важен формат записи строк — напрямую коды символов или непосредственно символы.

Как записать строку "\u" ? Для этого используется механизм экранирования, т. е. запись символа \ перед каждой специальной последовательностью. Т.е. записать такую строку можно следующим образом: "\\u";

Работа со строками

Конкатенация	<code>str.concat("!")</code>
Длина строки	<code>str.length()</code>
Деление строки по критерию	<code>str.split(", ")</code>
Получение символа по индексу	<code>str.charAt(5)</code>
Проверка наличия подстроки	<code>str.contains("мир")</code>
Проверка пустой строки	<code>str.isEmpty()</code>
Замена символов строки	<code>str.replace("Привет", "Здоров")</code>
Усечение пробельных символов	<code>str.trim()</code>
Вычленение подстроки	<code>str.substring(0, 5)</code>
Получение массива символов	<code>str.toCharArray()</code>

```
String str = "Привет, мир";
```

17

Notes

В таблице приведены лишь некоторые функции для работы со строками, подробнее о доступных возможностях работы со строками можно найти в документации по типу String. В рассматриваемом примере была объявлена строка `str`. Рассмотрим несколько примеров работы с объявленной строкой:

```
String total = str.concat("!"); // total = Привет, мир!  
  
total = str.substring(0, 5);    // total = Приве  
  
String[] parts = str.split(","); // parts = {"Привет" "мир"}
```

Индексы символов в строке начинаются от нуля. Т.е. индекс первой буквы = 0, второй = 1 и т. д.

Работа со строками



```
double x = Double.parseDouble(str);
```

```
-----  
if (str.matches("-?\\d+(\\.\\d+)?")) { // регулярные выражения  
    Double.parseDouble(str);  
}
```

```
-----  
for(char c : str.toCharArray()) {  
    // ручная проверка каждого символа  
}
```

```
-----  
try {  
    double d = Double.parseDouble(str);  
    System.out.println("Input " + d);  
} catch (Exception e) {  
    System.out.println();  
}
```

20

Notes

Стоит немного остановиться на задании для первой лабораторной работы. В рамках этой работы предлагается вводить аргументы для вычисления математической функции через аргументы командной строки о которых мы поговорим подробнее на следующих слайдах. Сейчас достаточно понять, что аргумент пользователь вводит в программу в виде строки, а для вычисления математического выражения необходимо число. Как решить эту проблему?

Среди прочих средств, в Java есть механизм разбора строки в число заданного типа. В случае, если в аргументе ожидается вещественное число, можно использовать следующую конструкцию для подобного преобразования:

```
double argument = Double.parseDouble ( str );
```

Но в случае, если переданный аргумент будет содержать НЕ вещественное число, то программа завершится с ошибкой. На слайде представлено три способа избежать подобной ошибки:

- перед разбором строки, проверить ее на соответствие «числовому шаблону»
- перед разбором строки, проверить каждый символ строки на соответствие требованиям
- используя синтаксическую конструкцию try-catch «отловить» полученную ошибку и реализовать обработчик этой ошибки

Форматированный вывод

```
double x = 5.683;
```

```
double y = Math.pow(x, 2.5);
```

```
System.out.printf("%.2f ^ 2.5 = %.3f", x, y);
```

Вывод: 5,68 ^ 2.5 = 76,890

18

Notes

Итак, мы можем рассчитать некоторые значения, используя математические функции и операции, можем объявлять и работать со строками. Однако остается вопрос: как правильно и удобно вывести результаты работы программы. Мы могли бы использовать несколько раз конструкцию `System.out.print` для вывода некоторых значений, но это будет не только утомительно, но и не удобно, как для программиста, так и может быть неудобным для пользователя программы. Для этого в Java есть возможность форматировать вывод программы. Для форматированного вывода используется конструкция `System.out.printf`.

Рассмотрим пример представленный на слайде. Есть две переменные: аргумент функции и ее результат. Хотелось бы вывести пользователю программы целиком формулу, которая была рассчитана программой. На слайде мы можем увидеть, что конструкция форматированного вывода принимает три аргумента: строка формата, и переменные, значения которых необходимо подставить в заданную строку в определенном виде.

Рассмотрим подробнее строку формата. Ключевым символом в строке в место которого будет подставлено значение является символ процента (%). Далее за ним следуют опциональные значения, задающие формат вывода аргумента. В данном случае опции первого аргумента имеют следующие значения: `.2f`

Это означает, что аргумент в данном месте строки стоит подставить в виде вещественного числа с двумя знаками после точки — символа разделяющего дробную и целую часть.

Глупо знать и не делать. Опасно делать и не знать.

```
1. public class PracticValiables {
2.
3.     public static void main(String[] args) {
4.
5.         byte x = 127;
6.
7.         x++;
8.
9.         System.out.printf(x);          //   -128
10.
11.        final String eyesColor = "green";
12.
13.           // eyesColor = "";
14.
15.        int size = 5;
16.
17.           size = (int)Math.exp(size);
18.
19.           size = (int)Math.exp(Math.E);
20.
21.    }
22. }
```

22

Notes

Вопросы для самопроверки:

0. Что произойдет, если удалить символы начала комментария на 13-ой строке?
1. Объясните почему операция инкремента на 7-ой строке привела к результату -128?
10. Для чего нужно приведение результата округления к типу int ?

Продолжение

```
1. public class PracticMath {  
2.  
3.     public static void main(String[] args) {  
4.  
5.         double x = 5.1, y = 3.57;  
6.  
7.         double res = Math.sin(( x + 1) / 3*Math.PI) * 8*Math.cos(y);  
8.  
9.     }  
10.  
11. }
```

$$\sin \left(\frac{x + 1}{3\pi} \right) \times 8\cos(y)$$

Notes

Пример реализации математической формулы в Java.

Продолжение

```
1. public class PracticString {
2.
3.     public static void main(String[] args) {
4.
5.         String name = "Vladimir";
6.
7.         String subName = name.substring(0, 4);    // "Vlad"
8.
9.         String[] parts = name.split("i");        // { "Vlad" "m" "r" }
10.
11.         final String eyesColor = "green";
12.
13.         // eyesColor = "";
14.
15.         int result = (int) 3 * Math.PI / 2;
16.
17.         System.out.printf("3 * PI / 2 ~ %d", result);
18.     }
19. }
```

21

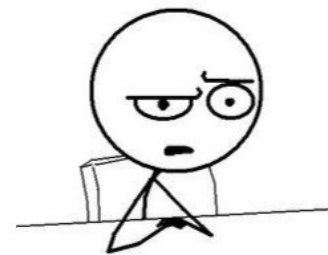
Notes

Вопросы для самопроверки:

1. Каков будет вывод после выполнения конструкции на 17-ой строке?
2. Найдите ошибку в 15-ой строке.
3. Чему будет равна переменная *subName*, если аргументы функции *substring* изменить на 1 и 5?

Аргументы командной строки

```
1. public class Lab0 {  
2.  
3.     public static void main(String[] args) {  
4.  
5.         if(args.length > 0) {  
6.             System.out.println ( args[0] );  
7.         }  
8.  
9.     }  
10. }
```



```
1. javac Main.java // запуск программы javac с аргументом  
2. java Main  
3. java Main 3 2 4
```

25

Notes

Согласно заданию, в первой лабораторной работе, аргумент для вычисления заданной функции необходимо считывать из аргументов командной строки. Что это за аргументы? Для того, чтобы понять концепцию передачи параметров запускаемой программе, рассмотрим набор команд, представленный в нижней части слайда.

В первой строке мы запускаем программу *javac* и далее указываем путь к файлу с исходным кодом. Пусть к файлу в данном случае будет передан как аргумент программе *javac*. Компилятор получит этот аргумент в виде строки (последовательности символов) и попытается открыть файл с указанным именем.

Параметров может быть более одного. В таком случае они разделяются пробелом(ми). Если передаваемый аргумент (строка) содержит в себе знак пробела, то такой аргумент обычно обрамляют двойными кавычками (зависит от ОС).

Аргументы командной строки

```
1. public class Lab0 {  
2.  
3.     public static void main(String[] args) {  
4.  
5.         if(args.length > 0) {  
6.             System.out.println ( args[0] );  
7.         }  
8.  
9.     }  
10. }
```

```
1. javac Main.java // запуск программы javac с аргументами  
2. java Main      // запуск программы java с аргументом  
3. java Main 3 2 4 // все аргументы после имени класса будут  
                  // переданы Java-программе
```

26

Notes

Во второй строке мы запускаем программу *java* (виртуальную машину Java) и в качестве аргумента указываем имя класса, который должен содержать точку входа в приложение — метод *main* с необходимыми модификаторами и сигнатурой. Чтобы передать нашей программе аргументы мы указываем их после имени класса. Рассмотрим, что при этом происходит.

Аргументы командной строки

```
1. public class Lab0 {  
2.  
3.     public static void main(String[] args) {  
4.  
5.         if(args.length > 0) {  
6.             System.out.println ( args[0] );  
7.         }  
8.  
9.     }  
10. }
```

```
java Main 3 2 4
```



27

Notes

Запускается виртуальная машина и загружает указанный класс (Main, код которого находится в файле Main.class в текущей директории). В данном классе она находит метод, который объявлен следующим образом:

public static void main (String[] args)

Этот метод принимает в качестве аргументов массив строк. Java-машина инициализирует этот массив, заполняя его теми аргументами, которые были указаны после имени класса Main. Этот процесс показан на слайде. Строки «3», «2» и «4» будут переданы в качестве аргументов нашей программе.

А как передать параметры непосредственно виртуальной машине? Все эти параметры указываются перед именем класса. Например, параметр отвечающий за максимальное количество оперативной памяти, которую может использовать виртуальная машина:

```
java -Xmx2G Main 1 2 3    // максимум 2 гигабайта
```

Classpath & Imports

```
> javac Main.java      // компиляция кода в Main.class  
  
> java Main 3 2 4      // запуск VM для исполнения Main  
  
  
  
  
  
  
  
> javac itmo/java/labs/0/Main.java // itmo/java/labs/0/Main.class  
  
> java Main      // error!!!  
  
  
  
  
  
  
  
> java -cp itmo/java/labs/0 Main
```

28

Notes

А как быть, если мы хотим запустить виртуальную машину находясь в директории, отличной от той, в которой находится Main.class? А если наш проект состоит из нескольких классов, которые находятся в разных директориях? Для решения этой проблемы есть соответствующий параметр виртуальной машины:

java -cp путь_к_классам Main

Если мы хотим пользоваться несколькими источниками для наших классов, то можно указать несколько путей, разделив их символом двоеточия.

Classpath & Imports

```
1. import static java.lang.Math.*;
2.
3. /**
4.  * Безысходники
5.  */
6. public class PracticMath {
7.
8.     public static void main(String[] args) {
9.
10.         double x = 5.1, y = 3.57;
11.
12.         double res = sin(( x + 1) / 3*PI) * 8*cos(y);
13.
14.     }
15.
16. }
```

29

Notes

Еще одной стороной использования других классов в своих программах является необходимость их «подключения» в своих программах. Для этого в вашем исходном коде указывается ключевое слово `import` с некоторыми параметрами.

В рамках первой лабораторной работы Вам предлагается реализовать математические вычисления, используя функции, объявленные в «библиотеке» (классе) `Math`. Для этого может быть использована конструкция `import`, представленная на слайде. Ее использование позволяет не указывать название `Math` перед каждым его использованием, а напрямую использовать математические функции, как это показано на слайде.



Кафедра вычислительной техники

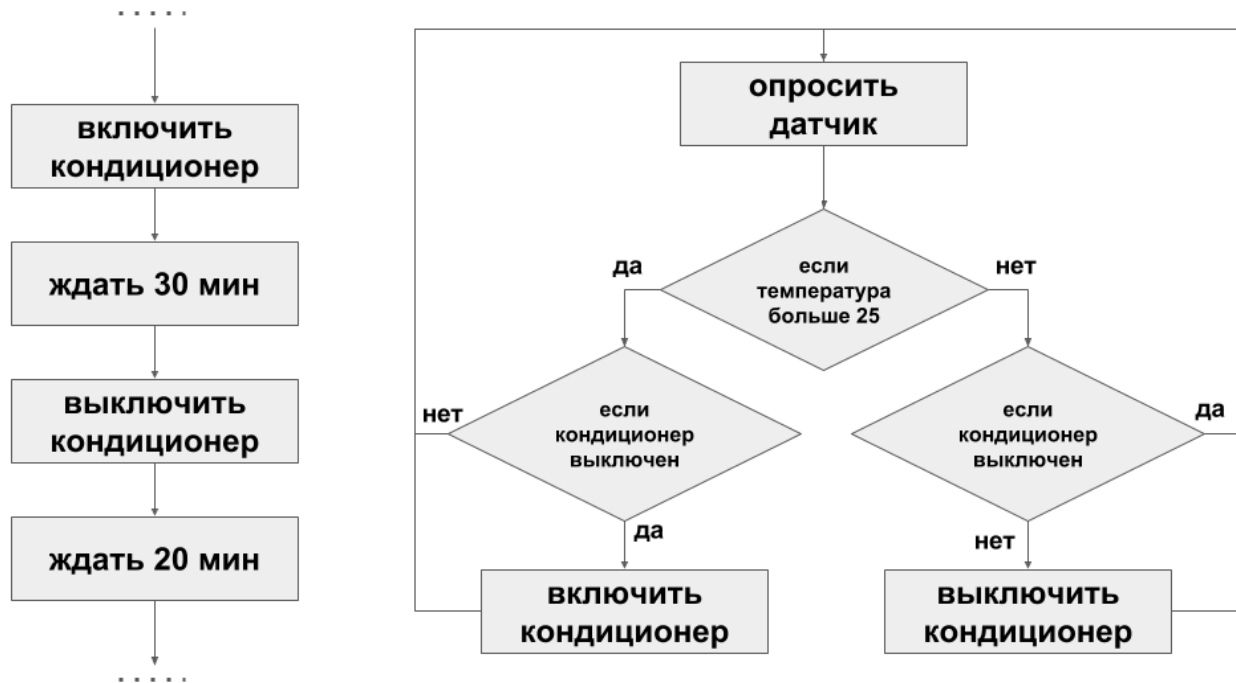
Программирование

Лекция #2.

Управляющие конструкции.

Подпрограммы.

Линейные и нелинейные программы



Notes

Программы можно (условно) разделить на две категории — линейные и нелинейные. Линейная программа предполагает последовательное выполнение набора операций. В то время, как нелинейная программа предполагает выполнение различного набора операций в зависимости от некоторых условий. Например, можно рассмотреть два алгоритма, представленных на слайде. Слева располагается алгоритм, который является линейным, в противовес ему, блок-схема справа предлагает нелинейный алгоритм.

В данной лекции мы рассмотрим какие в Java есть механизмы ветвления программ, организации циклов и т.д.

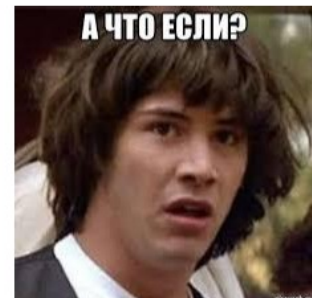
Условные выражения

`if ( условие ) выражение`

`if ( условие ) выражение else выражение`

`if ( условие ) выражение else if ( условие ) выражение ...`

```
1. final int LIMIT_TEMPERATURE = 25;
2.
3. int t = 21;
4. boolean isSwitchedOff = false;
5.
6. if(t > LIMIT) {
7.     ...
8. }
```



Notes

Первый инструмент — оператор *if*. Он необходим для выполнения некоторого числа операторов в зависимости от некоторых условий. Синтаксис оператора во всех его возможных вариантах представлен на слайде. Рассмотрим некоторые элементы этого оператора подробнее.

Условие — это выражение, тип результата вычисления которого является *boolean*, т. е. логический тип. Это говорит о том, что условием может быть любой логический оператор, операторы сравнения, а также переменные типа *boolean*. В примере, который представлен на слайде мы осуществляем вычисление выражение $t > LIMIT$, результатом которого является значение логического типа.

В зависимости от полученного в условии результата будет выполнено выражение, либо будет осуществлен переход на другую ветвь *else* оператора *if*, если таковая ветвь имеется.

Тернарный оператор

`условие ? выражение : выражение ;`

`// выражение обязательно должно возвращать значение`

```
int delta = x > 0 ? x : Math.abs(x) ;
```

```
1. int delta;
2. if(x > 0) {
3.     delta = x;
4. } else {
5.     delta = Math.abs(x) ;
6. }
```

Notes

Тернарный оператор — это оператор, использующий три аргумента для осуществления заданной операции. Что это за операция и какие необходимы аргументы?

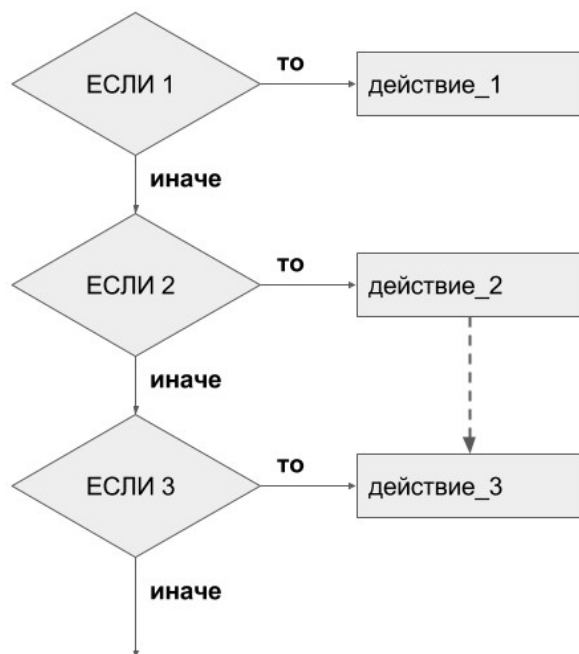
Синтаксис тернарного оператора имеет следующий вид:

`условие ? выражение_1 : выражение_2 ;`

В составе данного оператора, *условие* — это выражение, результат вычисления которого имеет тип *boolean*. Если результат вычисления условия принимает значение *true*, то выполняется *выражение_1*, иначе *выражение_2*. К указанным выражениям предъявляется требование — возвращать значение. Т.е. результат работы тернарной операции — это некоторое значение, которое, например, может быть присвоено переменной (как это показано в примере на слайде).

Соответственно, аргументами этого оператора являются: условие, *выражение_1* и *выражение_2*.

Многовариантное ветвление



```
switch ( x ) {  
  
case 1 : действие_1;  
        break;  
  
case 2 :  
case 3 : действие_2;  
  
default : действие_n;  
  
}
```

Notes

Оператор ветвления *switch* удобен в тех случаях, когда стоит сравнить некоторое значение с некоторым набором констант и в случае успешного сравнения, выполнить какое-либо действие. Блок схема работы оператора *switch* представлена на слайде.

Пример использования оператора *switch* представлен в правой части слайда. Давайте на этом примере рассмотрим механизм работы оператора ветвления.

В круглых скобках, стоящих сразу после ключевого слова *switch*, указывается имя переменной, которая может быть целочисленной, строковой либо являться перечислимый типом данных (о последнем речь пойдет в лекциях по темам ООП). Затем в фигурных скобках находится тело оператора, состоящее из блоков *case*. После ключевого слова *case* в каждом блоке указывается константа с которой следует сравнить переменную *x*. В случае успешного сравнения будет выполнено соответствующее действие. В конце блока может находиться ключевое слово *break*, которое в данном контексте инициирует выход из оператора *switch*. Если это ключевое слово опущено, то после выполнения действий в текущем блоке, управление будет передано действию, находящемуся в следующем блоке (в примере у нас таких блоков два: *case 2* и *case 3*).

В случае, если совпадения не нашлось и присутствует необязательный элемент *default*, то будут выполнены действия, объявленные внутри этого блока «по умолчанию».

Неопределенные циклы

```
while ( условие ) выражение;
```

```
while ( true ) {  
  
    // опрос датчика температуры  
    // вкл/откл кондиционера  
  
}
```

```
while ( x > 0 ) {      // может ни разу не выполниться  
  
    // code here  
  
}
```

Notes

Первый вариант неопределенного цикла представлен на слайде. Условие в данном случае — это такое выражение, которое мы использовали при описании оператора if.

Принцип работы данного цикла следующий: пока условие принимает значение true, выполняется выражение.

Неопределенные циклы

```
do выражение while ( условие );
```

```
do {  
  
    // опрос датчика температуры  
    // вкл/откл кондиционера  
  
} while ( true );
```

```
do {      // выполнится минимум один раз  
  
    // code here  
  
} while ( x > 0 );
```

Notes

Второй неопределенный цикл отличается тем, что сначала выполняется выражение после которого уже проверяется условие. Выражение будет повторяться до тех пор, пока условие возвращает значение true.

Определенный цикл for

```
for ( блок_инициализации; условие; блок_вычисления ) выражение;

/*
   вывод "арабских цифр"
*/
for (int i = 0; i < 10; ++i) {

    System.out.println( i );

}

// таблица квадратов
for (int i = 0, j = 0; i < 10 && j < 10; ++i, ++j) {

    System.out.println("%d * %d = %d", i, j, i * j );

}
```

Notes

Синтаксис определенного цикла представлен на слайде в первой строке. Разберем его составляющие:

Блок инициализации — это выражения в которых, как правило, объявляются переменные область видимости которых ограничивается этим циклом (т. е. вне цикла этими переменными нельзя пользоваться).

Условие — выражение, результат вычисления которого имеет логический тип.

Блок вычисления — это выражения в которых, как правило, происходит модификация объявленных в блоке инициализации переменных.

Цикл продолжает выполняться, пока условие возвращает true.

Итеративный цикл for

```
for ( объявление_переменной : множество ) выражение;  
  
String str = "some string";  
  
for (char currentChar : str.toCharArray()) {  
    if (currentChar != 's') {  
        System.out.print(currentChar);    // some string  
    }  
}
```

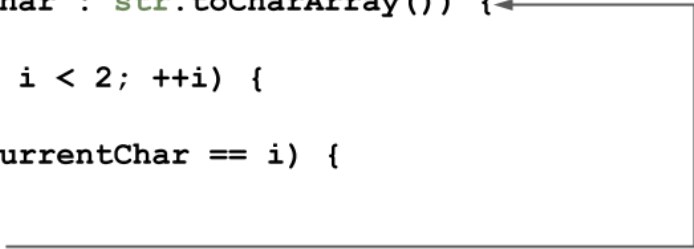
Notes

Итеративный цикл удобен в применении, когда необходимо перебрать набор каких либо значений не изменяя их. Подробнее этот цикл будет рассмотрен в разделе объектно-ориентированного программирования.

В примере на слайде данный цикл используется для перебора массива символов.

Прерывание потока управления

```
1. String str = "some string";
2.
3. for (char currentChar : str.toCharArray()) {
4.
5.     for(int i = 0; i < 2; ++i) {
6.
7.         if ((int)currentChar == i) {
8.
9.             break;
10.
11.         }
12.     }
13. }
```

A diagram consisting of a rectangular box with a line extending from the 'break;' statement on line 9 to the opening curly brace of the outer 'for' loop on line 3, illustrating that the break statement exits the inner loop and continues the execution of the outer loop.

Notes

Зачастую необходимо прервать выполнение цикла, не зависимо от условия, которое указывается в разных типах циклов. В таком случае в java имеется оператор break. Он прекращает выполнение цикла и передает управление выражению, следующему сразу за прерванным циклом.

В примере на слайде, оператор break передает управление внешнему циклу.

Прерывание потока управления

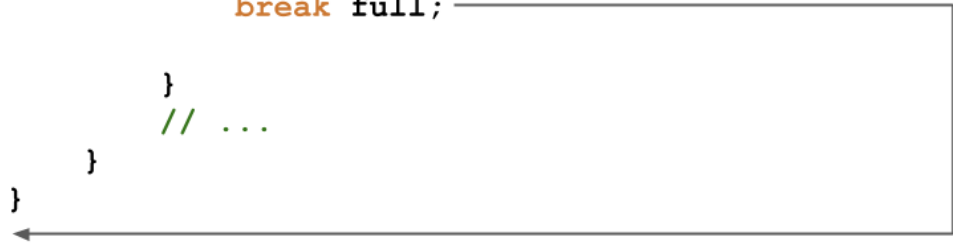
```
1. String str = "some string";
2.
3. for (char currentChar : str.toCharArray()) {
4.
5.     for(int i = 0; i < 2; ++i) { ←
6.
7.         if ((int)currentChar == i) {
8.
9.             continue;
10.
11.         }
12.         // ...
13.     }
14. }
```

Notes

Оператор `continue` необходим для того чтобы перейти к следующей итерации. Что это значит? При выполнении `continue` в неопределенных циклах, происходит вычисление условия и если оно возвращает `true`, то код цикла начинает выполняться снова. В определенном цикле сначала происходит выполнение в блоке вычислений, а затем в блоке условия. После проверки последнего цикл приступает к выполнению. В итеративном цикле происходит проверка: есть ли еще элементы и если есть, то начинается выполнение цикла со следующим элементом из набора.

Прерывание потока управления

```
1. String str = "some string";
2.
3. full:
4. for (char c : str.toCharArray()) {
5.
6.     for(int i = 0; i < 2; ++i) {
7.
8.         if ((int)c == i) {
9.
10.            break full;
11.
12.        }
13.        // ...
14.    }
15. }
```



Notes

А что если хочется выйти из всех вложенных циклов, как например в коде на слайде? В таком случае следует использовать оператор *break* с меткой. Для этого необходимо придумать и написать любой литерал перед объявлением цикла, затем поставить двоеточие и далее идет объявление цикла. В случае необходимости выйти из этого цикла нужно указать оператор *break*, после которого имя метки, которую мы уже поставили ранее.

Пример такого механизма представлен на слайде. В данном случае придуманный литерал — это «full».

Блоки и области видимости

```
{  
    // выражения, операторы и т.д.  
}
```

```
if ( условие ) выражение  —————→  if ( условие ) {  
                                         // код  
                                         }
```

```
public static void main (String[] args) {  
    // код  
    {  
        // блок  
    }  
    // код  
}
```

Notes

Область видимости переменной — это участок кода, в рамках которого эта переменная может быть использована. В java области видимости задаются фигурными скобками. В случае, если нужно ограничить видимость переменной, можно вставить в произвольную часть кода фигурные скобки, внутри которых и объявлять все необходимые переменные. Такая синтаксическая конструкция называется блоком кода.

“Подпрограммы” (методы)

```
public static void printMessage (String msg) {  
  
    System.out.println (msg);  
  
}  
  
public static void main(String[] args) {  
  
    printMessage ("я жидкий");  
  
}
```

Notes

Иногда возникает необходимость выполнять какую-либо последовательность операций повторно. В таких случаях избежать повторяющегося кода можно за счет использования *методов*. Метод — это своего рода подпрограмма, которая может принимать необходимые параметры и возвращать результат своей работы. Пока мы не изучили объектно-ориентированную модель языка *java* будем считать метод подпрограммой. Рассмотрим его синтаксис на примере, представленном на слайде.

В этом примере есть уже знакомый нам метод *main*, в котором мы вызываем наш собственный метод *printMessage* и в качестве аргумента предоставляем ему строку. Выше представлен этот метод. Сначала мы указываем два ключевых слова *public static*, затем указываем тип возвращаемого значения (в нашем случае метод ничего не возвращает и на месте типа указывается ключевое слово *void*). Далее указываем имя нашего метода и в круглых скобках указываем ожидаемые параметры, которые необходимо указать при вызове этого метода. После этого следует блок кода (см. предыдущий слайд) в котором находится реализация этого метода, т. е. алгоритм его работы.

Методы

```
public static int cube (int arg) {  
    return arg * arg * arg;  
}  
  
public static void main(String[] args) {  
    printMessage ("5 в кубе = " + cube(5));  
}  
  
public static void printMessage (String msg) {  
    System.out.println (msg);  
}
```

Notes

На слайде представлено несколько примеров различных методов и варианты их использования в методе *main*.

Стоит отметить, что в методе *cube* типом возвращаемого значения указан тип *int*, а в блоке кода присутствует ключевое слово *return*, после которого находится выражение, вычисляющее результат. После этого слова может быть указано любое выражение или переменная, значение которой соответствует типу возвращаемого значения. Полученный результат будет подставлен в место вызова метода *cube*.

При вызове метода все его параметры копируются, по этой причине, любое их изменение не затронет исходного значения, переданного в качестве параметра.

Методы с переменным количеством аргументов

```
public static void main(String[] args) {  
  
    printMessage ("Боромир", "мем");  
    printMessage ("Бор", "о", "мир", "^_^", "мем");  
}  
  
public static void printMessage (String ... msg) {  
    for(String s : msg) {  
        System.out.println (msg);  
    }  
}
```

Notes

Ранее мы рассматривали форматированный вывод. Метод которым мы в том случае пользовались вызывался так:

System.out.printf(...)

В качестве аргументов этот метод принимал шаблон форматирования и некоторое количество аргументов, которое нужно для этого формата. Однако как писать собственные методы, содержащие переменное количество аргументов? Для этого используется конструкция, представленная на слайде. Тип параметра и имя параметра в этом случае разделяются многоточием. Работать с такой переменной внутри метода нужно как с массивом.

Шаблон программы для л/р №2

```
1.  public class SortArray {
2.
3.      public static void main(String[] args) {
4.
5.          float[][] array = readArray ("...");
6.
7.          float[][] sortedArray = sort( array );
8.
9.          printResults ( array, sortedArray );
10.     }
11.
12.     public static void printResults (float[][] src, float[][] dest) {
13.         // your code here
14.     }
15.
16.     public static float[][] sort(float[][] array) {
17.         // your code here
18.     }
19.
20.     public static float[][] readArray (String pathToFile) {
21.         // your code here
22.     }
23. }
```

Notes

Зная каким образом могут быть организованы подпрограммы, Вам следует использовать организацию «модулей» программы, согласно шаблону, представленному на слайде. В данном примере каждая логически отдельная операция вынесена в отдельный метод.

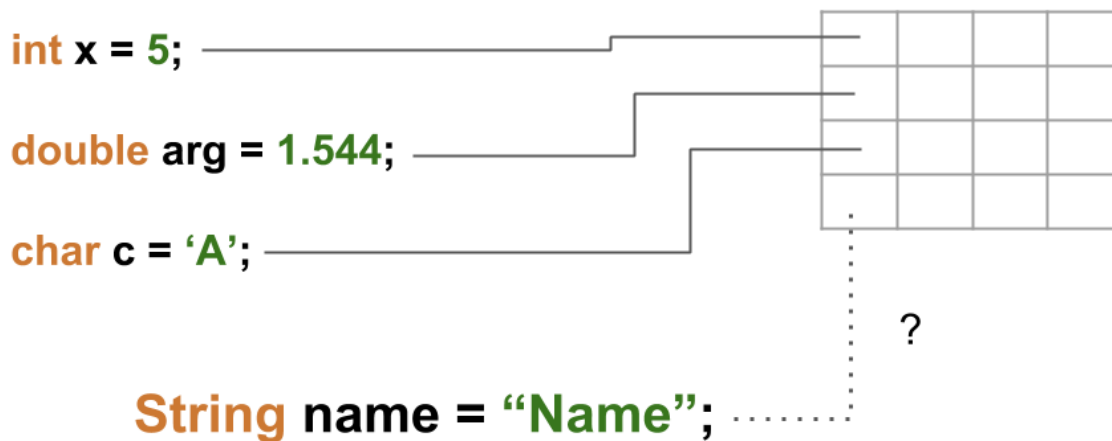


Кафедра вычислительной техники

Программирование

Лекция #3.
Массивы. IO.

Ссылочные и значимые типы данных



6

Notes

В Java все переменные делятся на два вида: ссылочные и значимые. Что это за виды переменных и в чем между ними разница?

Как мы рассматривали ранее, в Java есть некоторый набор примитивных типов данных и значения этих типов напрямую ассоциируются с переменными и JVM умеет оперировать их значениями, без каких-либо дополнительных действий. Но что если значение переменной очень велико и для его хранения нужно пространство в памяти гораздо большее, чем для примитивных типов?

Примером такого типа может быть String. На слайде представлен случай, когда значение переменной name — это набор из символов и для хранения этого значения нужно более одной ячейки памяти (ячейка памяти в данном примере — это условный объем памяти, которого достаточно для хранения примитива). Что происходит в этом случае?

Для этого можем рассмотреть абстрактный пример. Представьте, что у вас есть несколько пакетов, каждый из которых может поместиться в одну камеру для хранения, которые можно встретить при входе в супермаркет. Но что если у Вас есть коробка таких пакетов? Она естественно не влезит. В этом случае можно расфасовать пакеты из коробки по камерам и запомнить в каких находятся наши вещи.

Примерно тоже самое происходит в нашем примере на слайде. Примитив — это «пакет», который может поместиться в ячейку памяти, однако строка — более объемный объект и для ее хранения потребуется несколько ячеек.

Ссылочные и значимые типы данных

String name ————— **“Name”**

N	a	m	e

```
String name1 = new String(“Name”);
```

```
String name2 = new String(“Name”);
```

```
System.out.println ( name1 == name2 );
```

7

Notes

В таком случае мы разбиваем наш объект на фрагменты, складываем их в подряд идущие ячейки памяти и переменной *name* присваиваем *ссылку* на тот участок памяти, в котором мы сохранили наш объект. Таким образом, мы получаем ссылочную переменную (String в свою очередь можно назвать *ссылочным* типом данных).

В этом примере переменная *name* будет хранить только *ссылку* на значение и для оперирования значением, JVM необходимо будет предпринимать дополнительные операции по чтению значения и при записи его обратно в память.

Для все эти тонкости? Почему бы просто не писать программу не задумываясь об этом? В нижней части слайда представлен пример из трех строк, который выводит на экран строку *false*. Если Вам понятно почему, то можно смело переходить к следующему слайду.

Рассмотрим этот пример подробнее. В первых двух строках у нас объявляются переменные *ссылочного* типа, которым присваиваются *ссылки* на две различные области памяти, хотя и с одинаковым значением. Далее оператор `==` осуществляет сравнение *ссылок*, а НЕ ЗНАЧЕНИЙ. По этой причине результат работы этого примера — *false*.

Массивы



```
int[] a;
```

```
int b[];
```

```
int[] x = {5, 2};
```

```
int[] y = new int[2];
```

8

Notes

Первой темой этой лекции являются массивы, однако зачем мы затронули тему о ссылочных типах. Дело в том, что массив в Java — это ссылочный тип данных. И предлагаю рассмотреть все по порядку.

Массив — это некоторое множество значений, которые хранятся линейно в памяти (т. е. располагаются друг за другом, ячейка за ячейкой, а не в разброс). Доступ к этим значениям осуществляется через одну переменную. Для этого после имени переменной в квадратных скобках указывается индекс необходимого элемента. Индекс — это номер элемента в массиве. Нумерация начинается с нуля. Наглядная схема, демонстрирующая структуру массива представлена на слайде.

Объявить массив можно несколькими способами. Некоторые из них представлены на слайде. В первых двух строках объявляется переменная массива типа *int*. Скобки сигнализируют о том, что это переменная массива. Тип данных указывает на то, какого типа могут быть элементы, которые в нем хранятся. В этих первых двух строках только лишь резервируется ссылка на пока еще не созданный массив (эта ссылка пока не указывает ни на какую область памяти). Третья строка — это способ объявления массива, при котором сразу задаются его значения. Значения при этом указываются в фигурных скобках через запятую. Четвертая строка — это объявление массива и резервация памяти под его элементы (в данном случае под два элемента). После выполнения этой строки, все значения массива у будут равны нулю.

Оператор new

```
int[] y = new int[2];
```

```
String str = new String("я жидкий");
```



9

Notes

Многokrатно мы уже использовали ключевое слово *new*. Настало время выяснить, что это такое и для чего мы его использовали.

Оператор *new* резервирует необходимую для создания объекта память и возвращает ссылку на полученный объект. На слайде представлено два примера использования этого оператора. В первой строке оператором *new* резервируется память для массива из двух целочисленных элементов и ссылка на массив присваивается переменной *y*. Во второй строке происходит такая же последовательность, только для объекта строки.

Массивы

```
int[] x = {5, 2};
```

```
int count = x.length; // СВОЙСТВО
```

```
java.util.Arrays // работа с массивами
```

сортировка

поиск

копирование

10

Notes

Какие операции возможны с массивами? У переменной массива доступно свойство *length* (размер массива). Для доступа к этому свойству после имени переменной стоит поставить точку, после которой имя этого свойства. В дальнейшем в наших лекциях мы будем говорить о такой синтаксической конструкции, как о «доступе через точку».

В Java есть множество стандартных инструментов для работы с массивами и выполнения различных операций, представленных на слайде.

Массивы

сортировка

`Arrays.sort ( ... )`

`Arrays.parallelSort ( ... )`

поиск

`Arrays.binarySearch ( ... )`

копирование

`System.arraycopy ( ... )`

`Arrays.copyOfRange ( ... )`

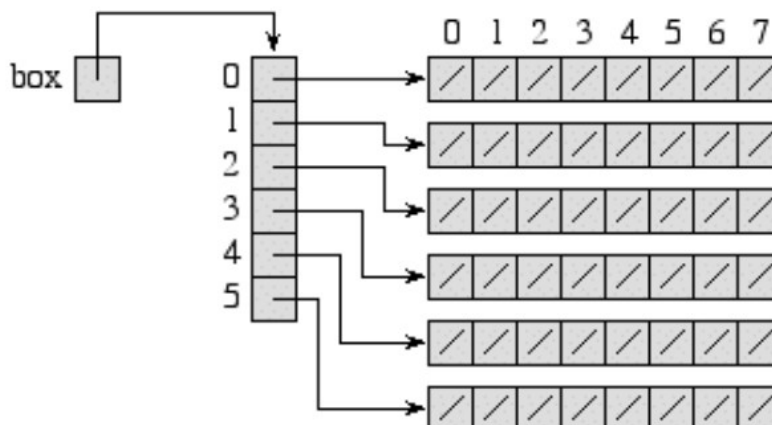
заполнение, применение заданной математической операции, установка значений по функции и т.д.

11

Notes

Синтаксические конструкции для применения различных операций представлены на слайде. Класс *Arrays* на самом деле содержит огромное количество методов, которые позволяют выполнить простые и сложные операции над массивом. Подробнее со списком этих методов можно ознакомиться в документации класса *Arrays*.

Массивы



```
int[][] matrix;
```

```
int[][] box = new int[6][8];
```

```
int[][] box = { {1, 2, 3}, {4, 5, 6} };
```

12

Notes

В Java имеется такой инструмент, как многомерные массивы. Это по факту массив, каждым элементом которого является другой массив, а у тех в свою очередь тоже может быть элементы равны другим массивам. Уровень вложенности массивов зависит от синтаксиса объявления, а именно, сколько скобок указано, такой и будет вложенность.

В примере на слайде представлены три варианта объявления массива. Первый вариант — резервирование ссылки на массив, но не инициализируя массив сразу. Второй вариант — это объявление массива и резервирование памяти под элементы. Эта строка соответствует картинке изображенной на слайде. И последнее — это объявление массива с инициализацией значениями. Механизм тут такой же, как и в случае с одномерным массивом. Для указания значений мы в фигурных скобках указываем их через запятую, а так как наши значения — это массивы, то указываем их и положено массивам — в фигурных скобках.

Стоит отметить, что во второй строке не все размерности могут быть указаны сразу. Например можно было указать только 6, тогда память зарезервируется для шести элементов массива, значения которых будут равны `null`. После чего можно инициализировать вложенные массивы следующим кодом:

```
for(int i = 0; i < box.length; ++i) {
    box[i] = new int[7 + i]; // вложенные массивы могут быть разной длины
}
```

Стандартные потоки

System.out
System.err
System.in

```
byte[] in = new byte[100];  
try {  
    System.in.read(in);  
} catch (IOException ioe) {  
    System.err.println("уПС");  
}  
String s = new String(in);  
System.out.println(s);
```

15

Notes

Многим уже знаком стандартный поток вывода. Однако в Java, как и везде в прочем, имеется три стандартных потока: ввода, вывода и ошибок. Доступ ко всем трем осуществляется через класс System. Пример работы со стандартными потоками представлен на слайде.

Ввод / вывод

IO	NIO
Потокоориентированный	Буфер-ориентированный
Синхронный (блокирующий)	Асинхронный
Один поток - один канал	Один поток - несколько каналов

16

Notes

В Java имеется (по факту) две реализации операций по вводу/выводу. IO — это первая, ~~пережившая~~ ~~многих~~ реализация, использующаяся по настоящее время. Основные ее характеристики представлены в таблице на слайде. Потокоориентированный подход заключается в том, что от источника данных к Вашему прикладному приложению организуется поток байт, либо символов и программа осуществляет их передачу последовательно. Для доступа к данным, которые находятся в конце файла придется подождать, пока будут считаны все предыдущие данные. То, что в IO подход синхронный говорит о том, что пока осуществляется операция ввода/вывода программа не может проводить какие-либо вычисления, пока ввод/вывод не завершится. Так, если Ваша программа считывает файл размером в пару гигабайт, то программа ощутимо зависнет на некоторое количество секунд. В NIO эти операции осуществляются асинхронно, к тому же подход в NIO является буфер-ориентированным. Т.е. Ваша программа посылает запрос, например, на открытие файла и продолжает выполнять какие-либо свои операции. В это время иницируется ввод данных из файла в некоторый буфер в памяти. По достижению окончания ввода Ваша программа может пользоваться буфером ввода/вывода, получая доступ к различным областям файла не перебирая все байты стоящие в файле до необходимого участка данных. В NIO имеется инструментарий для работы сразу с несколькими источниками данных при операциях ввода/вывода (Selectors). Несколько источников образуют каналы данных, которые могут переключаться с помощью селекторов (например, пока один источник ничего не передает).

Файлы в IO

```
File file = new File("путь_к_файлу");
```

```
file.canExecute();           // file.setExecutable(true);  
file.canRead();  
file.canWrite();  
file.exists();  
file.isDirectory();  
file.isFile();  
file.list();  
file.mkdir();
```



17

Notes

Название класса *File* в *io* несколько обманчиво, т. к. по факту класс предоставляет не работу с файлом (его содержимым), а только лишь дает различную информацию о нем. Если выполнить команду *dir* в командной строке windows или команду *ls -l* в терминале *linux/unix*, то можно увидеть массу атрибутов файлов, начиная от прав доступа и заканчивая датой последней модификации. По факту, именно этой информацией можно оперировать, используя класс *File*. Но также доступны и методы для создания директорий и изменения прав доступа.

Файлы в NIO

```
import java.nio.file.*;
```

```
Files.find( ... );
```

```
Files.probeContentType( ... );
```

```
Files.newOutputStream( ... );
```

```
byte[] fileContent = Files.readAllBytes(Paths.get("x.txt"));  
String content = new String ( fileContent );
```

18

Notes

Работа с файлами в NIO осуществляется средствами классов Files и Paths. Класс Files содержит набор методов, доступных через точку (пока так это назовем), которые имеют все функции, которые есть в `io.File`, а также набор дополнительных функций для просмотра атрибутов файлов и т. п.

С помощью класса Files можно также считать все байты. А затем построить из этих байтов строку. При создании строки можно также указать кодировку считанного текста.

java.io.Console

```
import java.io.*;
```

```
Console cli = System.console();
```

```
char[] password = cli.readPassword();
```

```
String str = cli.readLine();
```

```
cli.printf( ... );
```

```
// + форматированный ввод паролей
```

```
// + форматированный ввод строк
```

19

Notes

Console — это инструмент, удобный для интерактивного взаимодействия с пользователем, используя текстовый интерфейс. Так, например, с помощью «консоли» можно запросить ввод пароля (т. е. символы пароля не будут отображаться при вводе). В основном используется для ввода строк, учитывая определенный ожидаемый формат.

java.util.Scanner

```
Scanner scan = new Scanner(System.in);  
Scanner scan = new Scanner(new File("путь_к_файлу"));  
Scanner scan = new Scanner("строка с данными");
```

```
String notInt = scan.findInLine("[+]?\d+\\.?\d+");  
scan.hasNext(); // есть ли еще данные  
scan.hasNextDouble(); // есть ли вещественные  
scan.nextLine(); // считывание следующей строки
```

20

Notes

Scanner — удобный и полезный инструмент для работы с любым текстовым источником, как стандартным потоком или файлом, так и с переменной типа *String*. Этот инструмент позволяет анализировать входной поток символов на наличие тех или иных данных и предоставлять к ним доступ. Например, в нижней части слайда представлены четыре примера использования *Scanner'a*:

1. Поиск подстроки по заданному шаблону (в нашем случае — это простой шаблон вещественного числа). В случае успеха метод *findInLine()* вернет найденную подстроку.
2. Проверяет источник данных на наличие чего-либо еще (символов). Источником данных может быть все что указано в верхних строках слайда (поток, строки и т.д.)
3. Проверяет, есть ли вещественное число в источнике данных.
4. Считывает и возвращает следующую строку.

Это лишь не многие примеры функций, которые доступны у *Scanner'a*. Подробнее можно ознакомиться с ними в документации к классу *Scanner*.

java.io.BufferedReader

```
FileReader reader = new FileReader("filename");
BufferedReader br = new BufferedReader(reader);
String str = br.readLine();      // остановимся на null
if (str != null) {
    System.out.println("я не жидкий");
}
```

21

Notes

BufferedReader — это реализация так называемого буферного ввода-вывода в io. Он является некоторой надстройкой над потоком, чтобы предварительно собирать данные в буфер. В принципе, сам инструмент как таковой не имеет особых функций, но на примере, представленном на слайде, мы разберемся с одним из «подводных камней», встречающихся повсеместно. Речь пойдет о «пустых ссылках».

Чтобы понять что это такое, представьте, что вы объявили строку, но ничего ей не присвоили. Чему строковая переменная будет равна в таком случае? Как далее в программе проверить было ли ей присвоено значение или нет? Для этого есть ключевое слово *null*, которое и обозначает такую переменную. Т.е. для проверки строки из этого примера можно воспользоваться следующим кодом:

```
if (str == null) {
    System.out.println ( "Строка пустая" );
}
```

Код на слайде демонстрирует случай, когда метод *readLine()* возвращает такое значение (*null*). Отсутствие проверки на *null* может привести к исключительной ситуации *NullPointerException*.

ВНИМАНИЕ! РАЗДЕЛ ЕЩЕ РЕДАКТИРУЕТСЯ! ПОЖАЛУЙСТА, ПОДОЖДИТЕ!



Кафедра вычислительной техники

Программирование

Лекция #4.
Введение в ООП.

ООП... Что это?

- Общество Охраны Природы
- Объектно-Ориентированное Программирование
- Орган Опеки и Попечительства
- Организация Освобождения Палестины
- Основная Образовательная Программа
- Охрана Общественного Порядка
- Общество Отъявленных Птицеловов

Notes

ООП - общероссийская, общественная и культурно-просветительская экологическая организация объектно-ориентированное программирование.

ООП... Зачем это?

- Процедурное программирование
- Автоматное программирование
- Событийно-ориентированное программирование
- Логическое программирование
- Функциональное программирование
- Объектно-ориентированное программирование

Notes

набирает сообщение...

Что ты такое, ООП?



Notes

Синяя таблетка — проснешься и продолжишь писать процедурные программы и забудешь эту лекцию. Красная таблетка — ключ к познанию ООП.

Объекты реального мира...

...например, люди

```
public class Main {  
  
    public static void main(String args[]) {  
  
        String name = "Витяй Четкий";    // имя  
        int health = 625;                // здоровье  
        int mana = 820;                  // авторитет  
        int age = 28;                    // возраст  
        String skill = "Отжим мобилы";  // навык  
  
    }  
}
```

Notes

Реальные пример — программа эмулирующая поведение реального объекта, например, человека. В переменных вы храните параметры объекта.

Объекты реального мира умеют...

```
public class Main {  
  
    // main пропущен  
  
    public static void printProfile(...) {}  
  
    public static void sayPhrase() {  
        System.out.println("Дай мобилу погонять, э");  
    }  
  
    public static boolean assignPhone(int mana, int age) {  
        return mana > 800 && age > 15 && age < 35;  
    }  
}
```

Notes

В методах поведение.

Объекты реального мира умеют...

```
public class Main {  
    public static void main(String args[]) {  
        // объявление переменных пропущено  
        printProfile(name, health, age, skill);  
        sayPhrase();  
        if( assignPhone(mana, age) ) {           // если забрал  
            mana++;  
        }  
    }  
}
```

Notes

И кажется, что все прекрасно.

А что если их двое?



```
String name = "Витяй Четкий";    // имя
int health = 625;                 // здоровье
int mana = 820;                   // авторитет
int age = 21;                     // возраст
String skill = "Отжим мобилы";   // навык

String name2 = "Димас Рыжий";    // имя_второго
int health2 = 500;                // здоровье_второго
int mana2 = 747;                  // авторитет_второго
int age2 = 24;                    // возраст_второго
String skill2 = "Пикап мастер";  // навык_второго
```

Notes

А что если... Нужно сделать тоже самое для двух объектов? Как вариант реализовать набор переменных под характеристики другого человека.

А что если их двое?

// тут алгоритм для каждого свой

```
public static void sayPhrase() {  
    System.out.println("Дай мобилу погонять, э");  
}
```

// тут одинаковый алгоритм

```
public static boolean assignPhone(int mana, int level) {  
    return mana > 800 && level > 80;  
}
```

А что если их много?

Notes

И реализовать методы, которые нужны для каждого свои. Но как быть, если ту же задачу Вам поставят для миллиона человек?

Подход первый

```
String name[] = { ... };           // имена
int health[] = ...;                // здоровье
int mana[] = ...;                  // авторитеты
int age[] = ...;                   // возраста
String skill[] = { ... };          // навыки

public static void sayPhrase1() { }
public static void sayPhrase2() { }
// ...
public static void sayPhraseN() { }
```

А что, если у каждого массив навыков?



Notes

Выпив синюю таблетку и поспав, Вы можете продолжить в том же духе...

Подход второй

// собственный тип данных

class BadBoy {

```
String name = "Витяй Четкий"; // имя
int health = 625;                // здоровье
int mana = 820;                  // авторитет
int age = 21;                    // возраст
String skill = "Отжим мобилы"; // навык
```

}

// Терминология:

// класс

// поле

// синтаксис объявления



Notes

Однако здравствуйте, можно иначе. Собственные типы данных — первое ~~что мы~~ видим после лекарственных препаратов красного цвета с чем мы познакомимся и что является фундаментальным понятием в ООП.

Класс задает структуру и поведение

// собственный тип данных

```
class BadBoy {
```

// объявление остальных полей опущено

```
String message = "Дай мобилу погонять, э";
```

```
public void sayPhrase() {  
    System.out.println(message);  
}
```

```
public boolean assignPhone() {  
    health--;  
    return mana > 800 && level > 80;  
}  
}
```

Notes

Теперь присваивание телефона отнимает здоровье.

Как этим пользоваться

// содержимое main

// экземпляр класса

```
BadBoy boy = new BadBoy();
```

```
boy.health = 1; // обращение к атрибутам
```

```
boy.mana = 900;
```

```
boy.sayPhrase ();
```

```
if ( boy.assignPhone() ) {
```

```
    System.out.println("пасаны, зацените че");
```

```
}
```

Notes

Комментарий к этому слайду пока отсутствует

Цикл создания экземпляра класса

КОНКРЕТНЫЙ ПАЦАН



Основы программирования. ИТМО ВТ.

АБСТРАКТНЫЙ ПАЦАН



18

Лекция #4

Notes

Тут пока можно и без комментариев

Конструкторы

// содержимое main

// создаем экземпляр класса

```
BadBoy boy = new BadBoy();
```

// все здорово, но хочется как то сразу указать состояние

// например, значение поля *skill*

```
BadBoy boy = new BadBoy("Синхронный ввод/вывод");
```

Что для этого нужно?

Notes

набирает сообщение...

Конструкторы

```
class BadBoy {  
  
    String name = "Витяй Пикапер"; // имя  
    int health = 625;                // здоровье  
    int mana = 820;                  // авторитет  
    int age = 21;                    // возраст  
    String skill;                    // навык  
  
    public BadBoy() {  
        skill = "ой все";  
    }  
  
    public BadBoy(String sk) {  
        skill = sk;  
    }  
}
```

Notes

набирает сообщение...

Конструкторы

// содержимое main

// создаем экземпляр класса

```
BadBoy firstBoy = new BadBoy();
```

```
BadBoy secondBoy = new BadBoy("Синхронный ввод/вывод");
```

```
BadBoy [ ] group = new BadBoy[42];
```

```
for(int i = 0; i < 42; ++i) {
```

```
    group[i] = new BadBoy("Прост");
```

```
}
```

Notes

набирает сообщение...

Нужно больше параметров

```
// public BadBoy() {  
//   skill = "ой все";  
// }
```

```
public BadBoy(String sk) {  
    skill = sk;  
}
```

```
public BadBoy(String nm, String sk) {  
  
    skill = sk;  
    name = nm;  
}
```

Notes

набирает сообщение...

Описание класса в UML

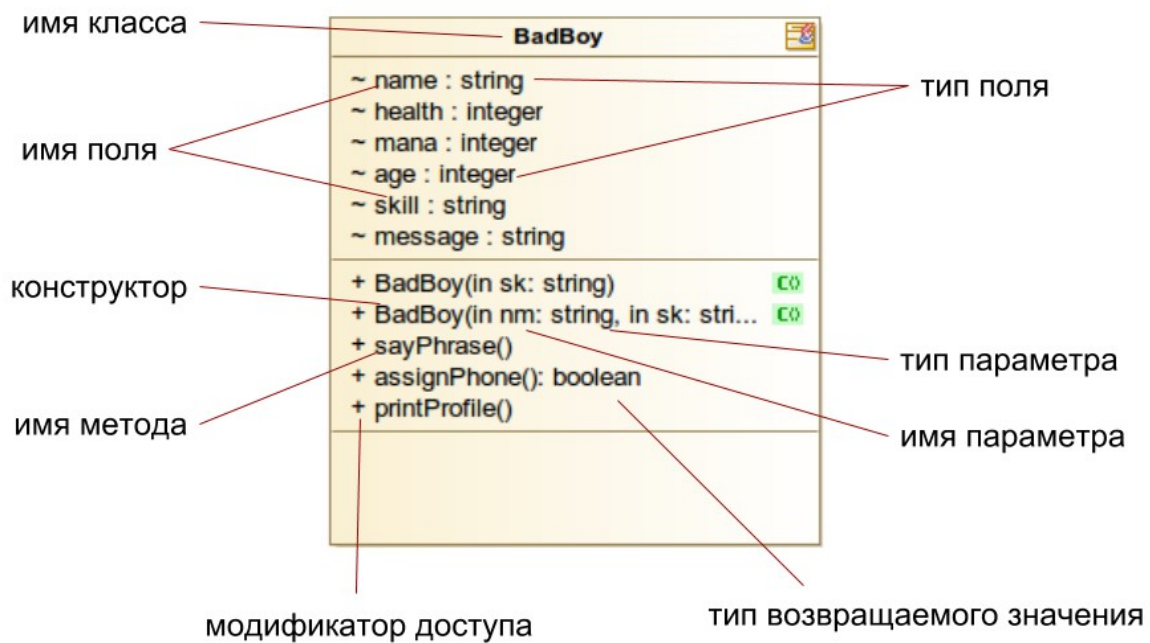
Немного википедии

UML (англ. Unified Modeling Language — унифицированный язык моделирования) — язык графического описания для объектного моделирования в области разработки программного обеспечения.

Notes

набирает сообщение...

Описание класса в UML



Notes

набирает сообщение...

Использование своих классов

```
import ? ;
```

```
public static void main(String[] args) {
```

```
    BadBoy boy = new BadBoy();
```

```
}
```

01) Для чего вообще писать import?

10) Откуда берутся названия java.io, java.util и другие?

Notes

набирает сообщение...

Пакеты

```
package ru.ifmo.se.prog.examples;  
  
public class BadBoy {  
  
    // все поля и методы класса  
  
}
```



Как это использовать?

Notes

набирает сообщение...

Пакеты

```
import ru.ifmo.se.prog.examples.BadBoy;  
  
public class Main {  
  
    public static void main ( String[] args ) {  
  
        BadBoy veryBadBoy = new BadBoy();  
  
        // ...  
    }  
}
```

А класс Main... он в каком пакете?

Notes

набирает сообщение...

Пакеты

```
package ru.ifmo.se.prog;  
  
import ru.ifmo.se.prog.examples.BadBoy;  
  
public class Main {  
  
    public static void main ( String[] args ) {  
  
        BadBoy veryBadBoy = new BadBoy();  
  
        // ...  
    }  
}
```

Notes

набирает сообщение...

Введение в историю мифологии



Notes

набирает сообщение...

Наследование

Что общего между двумя объектами реального мира?



Основы программирования. ИТМО ВТ.



Лекция #4

Notes

набирает сообщение...

Наследование

Поиск одинаковых атрибутов

- имя
- возраст
- рост
- вес
- пол

Поиск одинакового поведения

- сказать фразу
- применить навык
- присесть
- привстать

Notes

набирает сообщение...

Наследование

```
package ru.ifmo.se.prog.examples;

public class Human {

    String name;           // имя
    boolean isMale;        // пол. true если муж.
    int age;               // возраст
    // ... остальные свойства

    public void sayPhrase() { }
    public void applySkill() { }
    // ... остальные методы

}
```

Notes

набирает сообщение...

Наследование

```
package ru.ifmo.se.prog.examples;  
  
public class BadBoy extends Human {  
  
    public BadBoy() { isMale = true }  
  
    public void applySkill() {  
        assignPhone();  
    }  
  
    public void assignPhone() { ... }  
}
```

Notes

набирает сообщение...

Наследование

```
package ru.ifmo.se.prog.examples;  
  
public class BadGirl extends Human {  
  
    public BadGirl() { isMale = false }  
  
    public void applySkill() {  
        System.out.println("идет создание сэлфи");  
    }  
}
```

Notes

набирает сообщение...

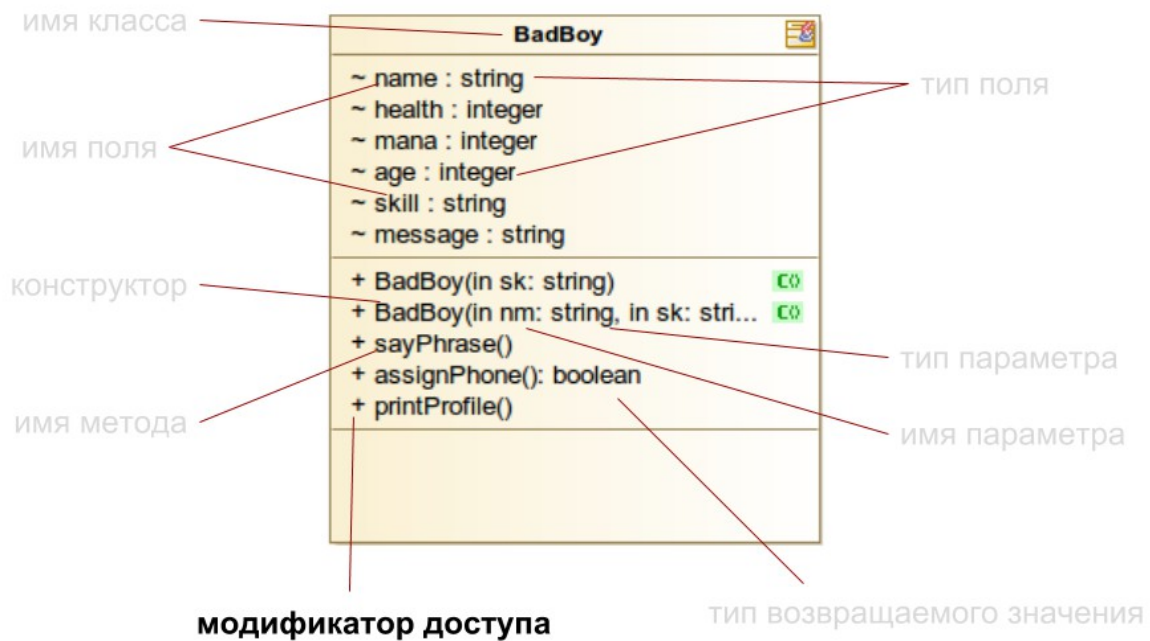
Наследование

```
package ru.ifmo.se.prog;  
  
import ru.ifmo.se.prog.examples.*;  
  
public class Main {  
  
    public static void main(String[] args) {  
        Human[] array = new Human[2];  
        array[0] = new BadBoy ();  
        array[1] = new BadGirl ();  
  
        for(Human human : array) {  
            human.applySkill ();  
        }  
    }  
}
```

Notes

набирает сообщение...

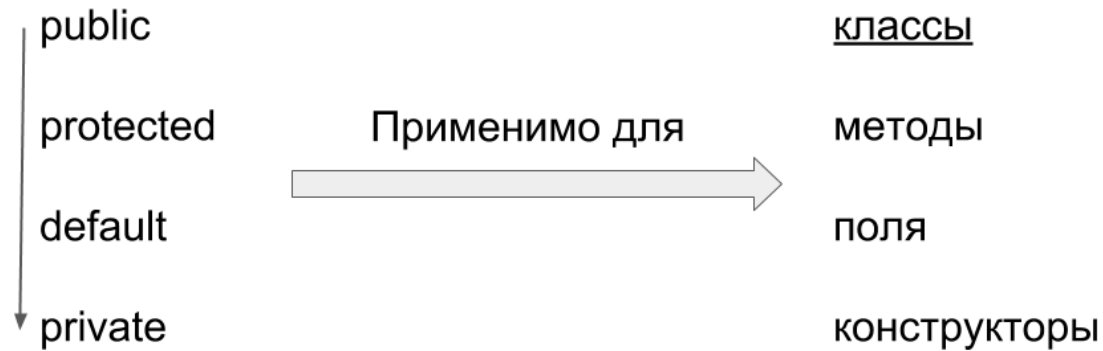
Описание класса в UML



Notes

набирает сообщение...

Модификаторы доступа



Notes

Дай конспект погонять.
Продолжение следует...

Зачем мы использовали `static`

Что есть `this` и `super`

Наследование конструкторов

Статические и не статические блоки инициализации



Кафедра вычислительной техники

Программирование

Лекция #5.
Погружение в ООП.

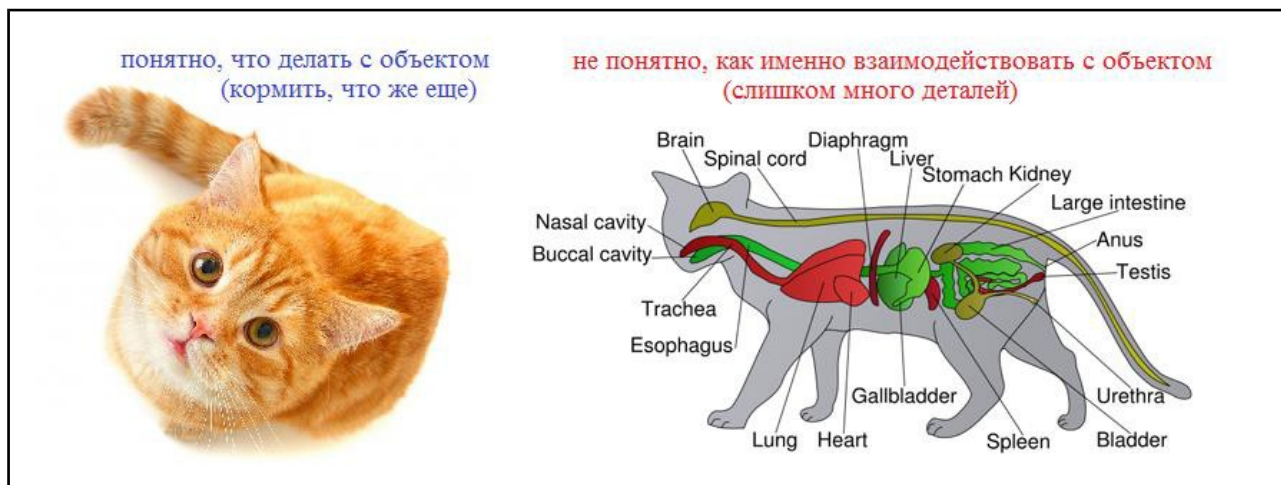
Полиморфизм

1. Переопределение методов
2. Перегрузка перегрузка
3. Коварианты возвращаемых типов

Notes

Комментарий еще не готов

Инкапсуляция



Notes

Наглядность инкапсуляции зашкаливает

Интерфейсы

1. Поведение объектов
2. Реализация интерфейса
3. Зачем интерфейсы, если есть наследование?

Notes

Комментарий еще не готов

Еще немного непонятных вещей.

1. Тип ссылки
2. Тип объекта
3. Явное приведение типов
4. Проверка типов объектов: instanceof
5. Проверка типов объектов: class (ДЗ)

Notes

Рассмотрим пример:

```
MySuperClass x = new MyClass();
```

В данном случае тип ссылки `MySuperClass`, а тип объекта `MyClass` — это тип той ссылки, которую вернет оператор `new`. Т.е. тип объекта под который фактически будет зарезервирована память.

Object

1. hashCode
2. equals
3. clone
4. finalize
5. clone
6. toString

Notes

Комментарий к слайду еще не готов



Кафедра вычислительной техники

Программирование

Лекция #6.

ОО - объектно-ориентированное

П - проектирование

Абстрактные классы

interface



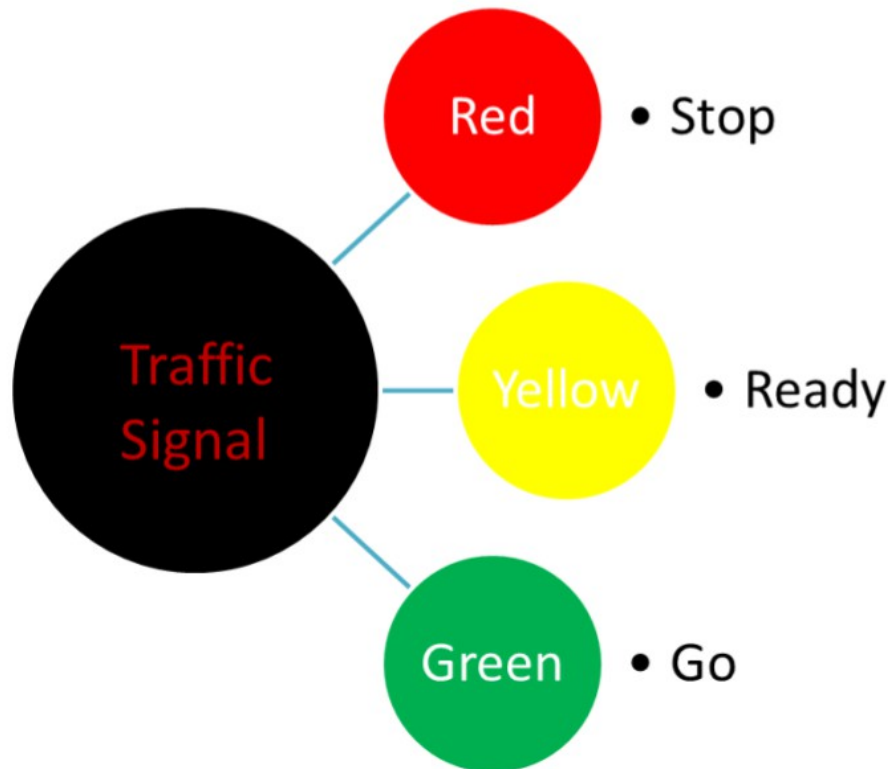
class

еще один
уровень
абстракции?

Notes

Комментарий к слайду еще не готов

Class, interface... enum



Notes

Перечисления в Java — это сложная и одновременно с тем, интересная штука. Перечисления позволяют реализовать заданное количество экземпляров некоторого класса, ограничив при этом программиста в создании этих экземпляров вручную. Рассмотрим пример на слайде. Имеется светофор. Каждый его сигнал — это экземпляр класса СигналСветофора. Однако мы знаем, что плодить объекты в данном случае не является необходимостью, т. к. объектов всего три: красный, желтый и зеленый. Пример того, как такая конструкция выглядит представлена ниже. Перечисления могут реализовывать интерфейсы. Для большего интереса этот момент рекомендуется к самостоятельному изучению.

CODE

```
public enum SignalTraffic {  
  
    RED, YELLOW, GREEN; // это будут экземпляры класса SignalTraffic  
  
    // тут могут быть ваши методы и поля. Конструкторы могут быть только private  
  
}
```

S.O.L.I.D. принципы

- Принцип единственной обязанности
- Принцип открытости/закрытости
- Принцип подстановки Барбары Лисков
- Принцип разделения интерфейса
- Принцип инверсии зависимостей

Notes

Рассмотрим эти принципы по порядку:

S — принцип единственной ответственности классов. Что это подразумевает? Это значит, что Ваши классы должны быть логически разделены и не содержать разнородной логики.

O — принцип закрытости данных классов для модификации и открытости для расширения.

L — принцип подстановки Барбары Лисков. Гласит, что при наследовании мы не должны менять поведение переопределяемых методов, чтобы использовать однотипно объекты всех дочерних классов, где ожидается родительский тип. Сложно написано, сори :(

I — это своего рода принцип единственной ответственности для интерфейсов. Не стоит придумывать интерфейсы, в которых сразу определены методы для всего. Разделяйте интерфейсы по логике их использования.

D — инверсия зависимостей, заключается в том, чтобы абстракции не зависели от реализаций, но чтобы зависимостями реализаций были абстракции. Коротко говоря: даешь больше интерфейсов!

S.T.U.P.I.D. принципы...

S - Синглтон

T - Сильная Связанность

U - Невозможность тестирования

P - Преждевременная оптимизация

I - Не дескриптивное присвоение имени

D - Дублирование кода

Notes

Думаю не особо правильно комментировать подробности неправильного программирования и оставляю их Вам для самостоятельного изучения.



Кафедра вычислительной техники

Программирование

Лекция #7.

Обо всем, что не сказано

Безударные гласные корня проверяются ударением, т.е. в неударяемом слоге пишется та же гласная, что и в соответствующем ударяемом слоге однокоренного слова.

ИСКЛЮЧЕНИЯ: пловец и пловчиха.

язык: русский .

Notes

Всюду есть правила. И без исключений из правил не обойтись...

```
MyClass instance = null;  
instance.toString();
```

ИСКЛЮЧЕНИЯ: NullPointerException.

язык: java .

Notes

Исключения в правилах языка не обошли и Java стороной. Исключения — это ситуации, которые можно характеризовать так: «Что то пошло не так». Например произошло деление на нуль и т. п.

На слайде приведен пример, когда мы пытаемся вызвать не статический метод у объекта, по null-ссылке. В этой ситуации Java пытается определить у какого объекта был вызван метод, не находит его и «бросает» исключение NullPointerException.

В отличии от русского языка, исключения в Java — это объекты определенного типа. Для работы с этими объектами в Java есть набор инструментов и языковых конструкций. К тому же сама Java-машина особым образом работает с объектами-исключениями.

На слайде приведен пример, который побудит виртуальную машину сгенерировать объект типа NullPointerException.

Классификация

- Error
 - VirtualMachineError
 - OutOfMemoryError
- Runtime (unchecked)
 - NullPointerException
 - IllegalArgumentException
- Checked exception

Notes

Все классы исключений можно классифицировать и пример такой классификации представлен на слайде.

Первый тип — это такие исключения, которые возникли по причине ошибок самой Java-машины, и Ваш код не может ничего с этим сделать. В качестве примера можно вспомнить ситуацию, когда в память, выделенная Java-машине закончилась. В такой ситуации нет смысла обрабатывать эту ошибку, разве что выводом записи на экран «Капут».

Второй тип ошибок — это ошибки времени выполнения, т. е. на момент компиляции еще не известно «вылетит» там ошибка или нет.

И третий вид ошибок — это ошибки времени компиляции о которых мы поговорим на следующем слайде.

Ключевые слова

```
public void method() throws IOException, IllegalArgumentException
{
    throw new IOException();
}

public void app() {

    try {

        method();

    } catch (IOException ex) {
        // кодЪ 1
    } finally {
        // кодЪ 2
    }
}
```

Notes

В этом примере у нас есть некоторый метод *method()*, который отмечен, как метод, способный сгенерировать исключения типов *IOException* или *IllegalArgumentException*. Такая конструкция обязывает программиста, который использует этот метод, обрамлять его конструкцией *try-catch*. Таким образом, в момент компиляции может быть проверено, обрабатывается ли ошибка указанная в определении используемого метода.

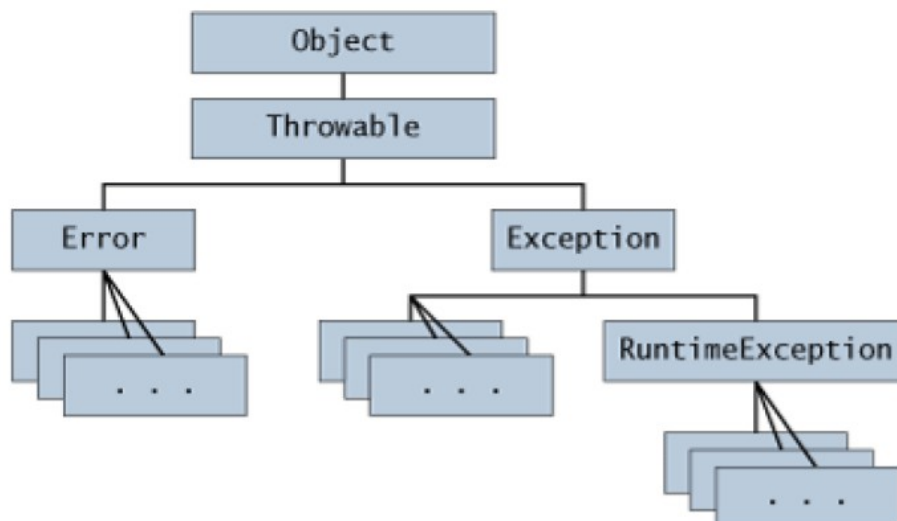
Для «отлавливания» и обработки исключений, используется конструкция *try-catch*. В блоке *try* находится код, который, возможно, генерирует исключения. В блоках *catch* указывается тип ошибки, которую обрабатывает этот блок *catch* и код-обработчик. В приведенном на слайде примере обработка исключений опущена. На ее месте находятся комментарии.

В конструкции *try-catch* есть еще один необязательный блок — *finally*. В теле этого блока прописывается код, который необходимо выполнить в любом случае, независимо от того, было ли сгенерировано исключение или нет.

Однако это не все, что касается исключений. Вы самостоятельно можете генерировать исключения, в надежде, что кто-либо «поймает» их. Это делается с помощью оператора *throw*. Пример генерации своего исключения представлен на слайде в методе *method()*.

Как сделать собственные исключения

```
public class MyException extends Exception { }
```



Notes

А как насчет собственных исключений? Можем ли мы генерировать и отлавливать исключения собственных типов? Да. Для этого достаточно определить свой класс, который наследуется от любого предка `Throwable` (см. иерархию наследования на слайде).

Все! Теперь Вы можете генерировать свое исключение следующим образом:

```
throw new MyException();
```

Стоит заметить, что в классе `MyException` Вы можете добавить каких угодно собственных методов, конструкторов, полей и т.п.

Multiple-catching

```
catch(...) {  
  
} catch (...) {  
  
}  
  
catch (IOException|SQLException ex) {  
    logger.log(ex);  
    throw ex;  
}
```

Notes

А что делать, если у нас одинаковый код в обработчиках catch для разных типов исключений? Для этого посмотрим на слайд и осознаем всю прелесть multiple-catch конструкции.

Анонимные классы

```
public void app() {  
  
    Program program = new Program() {  
  
        public void start() { /* кодЪ старт */ }  
  
        public void stop() { /* кодЪ стоп */ }  
    };  
}  
  
public interface Program {  
  
    void start();  
  
    void stop();  
}
```

РЕСПЕКТОВЫЕ РЕБЯТА



ПИШУТ АНОНИМНО

Notes

Ранее мы уже говорили о разных видах классов, однако есть еще некоторое количество их разновидностей, и один из таких мы рассмотрим на этом слайде.

Что происходит: мы объявили интерфейс Program и нам очень лень писать для него реализацию, т. е. создавать класс, который будет его реализовывать... Программеры вообще ленивые ребята, по сему для нас любимых есть возможность реализовать методы интерфейса на лету не заморачиваясь с новым классом.

Для этого мы пишем конструкцию, похожую на создание экземпляра типа Program, но прежде чем поставить завершающую точку с запятой... мы открываем блок кода в котором пишем реализацию методов этого интерфейса. Как на это реагирует наш компилятор?

Он генерирует за нас некоторый класс, имя которого мы не знаем, реализует в этом классе интерфейс Program, а тот код, что мы указали в блоке кода после вызова конструктора Program() Java вставит как содержимое этого класса, созданного «на лету». Сложновато. Но надо перечитать это несколько раз для большего понимания. По этому эту конструкцию и называют «анонимными классами», т. к. здесь представлен класс, но так вышло, что он без имени...

Куда, куда его можно вставить?

- в методе
- в блоке кода
- в блоке инициализации
- в блоке цикла
- в блоке условного оператора

(проще сказать куда нельзя)

Локальный класс

Notes

Самый странный в Java класс, который только может быть — это локальный класс. Такой класс можно объявить где угодно. И говоря где угодно имеется в виду любой блок кода. На слайде представлено лишь несколько примеров, где может быть объявлен такой класс.

Странность его в том, что объекты этого класса можно создать только в области видимости того блока, где он был объявлен. По этой причине не совсем понятно как он может быть использован на практике. Но такое тоже есть и об этом будет полезно знать.

Вложенный static class

- для создания объекта статического внутреннего класса не нужен объект внешнего класса
- из объекта вложенного класса нельзя обращаться к нестатическим членам внешнего класса

Notes

Комментарий к этому слайду еще не готов. Простите за предоставленные неудобства

CODE

```
public class Outer {  
  
    // поля и методы класса Outer  
  
    public static class Inner {  
  
        // поля и методы класса Inner  
  
    }  
}  
  
// Создание экземпляра вложенного класса:  
Outer.Inner instance = new Outer.Inner();  
  
// подключение вложенных классов:  
import pack.name.Outer.Inner;
```

Вложенный class

```
// создание экземпляра внутреннего класса
Outer outer = new Outer();
Outer.Inner query = outer.new Inner();

// Обращение к элементам внутреннего класса
Outer.this.element();
```

Notes

На слайде приведен образец создания экземпляра класса, определенного как вложенный класс.

CODE

```
public class Outer {

    // поля и методы класса Outer

    public void element() { }

    public class Inner {

        // поля и методы класса Inner

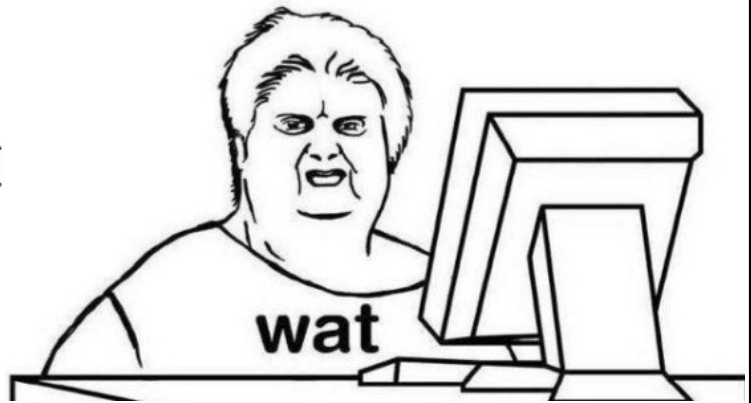
        public void someCode() {
            Outer.this.element(); // доступ к внешнему классу
        }

    }

}
```

Вопросы какие-то странные...

```
public void app() {  
    method(() -> System.out.println(""));  
}  
  
public void method(Program program) {  
    program.start();  
}  
  
public interface Program {  
    void start();  
}
```



Notes

Функциональное программирование — это интересно, но по настоящему Вы потрогаете его только в следующем семестре.

Вопросы для проверки

1. Что такое массив?
2. В чем отличие ссылочных типов данных от значимых?
3. Что такое итеративный цикл for?
4. Что такое класс? А объект?
5. Для чего нужны интерфейсы?
6. Что из себя представляет this?
7. В чем разница передачи параметров по ссылке и по значению?
8. Что такое исключения?
9. Можно ли не инициализировать константу при объявлении?
10. Как использовать стандартные потоки ввода/вывода в Java?
11. Что вы можете рассказать о конструкции: `for(;;) { }`
12. Как устроены многомерные массивы?
13. Можно ли создавать собственные исключения? Как?
14. Что из себя представляет переопределение и перегрузка методов?
15. Какие типы данных могут быть использованы в операторе switch?
16. Что такое интерфейс?
17. Сколько элементов может хранить массив?
18. Для чего нужен enum?
19. Как работает тернарный оператор?
20. Что такое null-ссылка?

Вопросы со звездочкой

1. Сколько элементов может хранить массив? Почему?
2. Как формируется hashCode по умолчанию?
3. Алгоритмы сборки «мертвых» объектов.
4. Где определен метод values() в enum'e?
5. Возможен ли вызов методов у null-ссылок?
6. Является ли Float.NaN типом Number?
7. Есть ли возможность поменять значение константы?
8. Для чего нужен метод getClass() определенный в классе Object?
9. В чем ошибка: `byte a = 1, b = 2; byte c = a + b;`
10. Как использовать функциональные интерфейсы?

